

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ярославский государственный университет имени П. Г. Демидова»

Кафедра дискретного анализа

Сдано на кафедру

«_____» _____ 2026 г.

Заведующий кафедрой,

д. ф.-м. н., доцент

_____ А. В. Николаев

Выпускная квалификационная работа

**Разработка игрового приложения
«Коридор» с модулем стратегического поведения
соперника**

по направлению

01.04.02 Прикладная математика и информатика

Научный руководитель

д. ф.-м. н., доцент

_____ А. В. Николаев

«_____» _____ 2026 г.

Студент группы ИВТ-21МО

_____ А. О. Карпунин

«_____» _____ 2026 г.

Ярославль, 2026

Реферат

Объем 93 с., 4 гл., 0 рис., 0 табл., 10 источников, 6 прил.

Ключевые слова: **игры, Коридор, поиск кратчайшего пути, игровой искусственный интеллект, Минимакс, альфа-бета отсечение, Монте-Карло, функция оценки, обучение весов.**

Объект исследования — логическая игра «Коридор» и алгоритмы автоматического расчета лучшего хода в ней.

Цель работы — разработка игрового приложения, с помощью которого пользователь сможет сыграть против одного из четырёх программных соперников, а также сравнить эти алгоритмы между собой.

Результат работы — приложение, реализующее правила игры «Коридор», с возможностью игры против компьютера или наблюдения за игрой алгоритмов друг против друга.

Содержание

Введение	5
1. Игра «Коридор»	7
1.1. Необходимые определения	7
1.2. Описание игры	7
1.3. Модуль стратегического поведения соперника	8
2. Алгоритмы	9
2.1. Общие функции и структура алгоритмов	9
2.2. Случайный алгоритм	13
2.3. Минимакс	14
2.4. Альфа-бета отсечения	16
2.5. Монте-Карло	17
2.6. Сводные параметры сложности	19
3. Игровое приложение	21
3.1. Техническая реализация	21
3.2. Взаимодействие с приложением	22
4. Сравнение алгоритмов	27
4.1. Методика сравнения	27
4.2. Собираемые метрики	27
4.3. План эксперимента	28
4.4. Форма представления результатов	28
4.5. Анализ результатов	29
4.6. Ограничения эксперимента	30
Заключение	31
Список литературы	32
Приложение А. Реализация интерфейса алгоритма	33
Приложение Б. Реализация случайного алгоритма	48
Приложение В. Реализация алгоритма MiniMax	52
Приложение Г. Реализация алгоритма AlphaBeta	60

Приложение Д. Реализация алгоритма Monte Carlo	74
Приложение Е. Реализация обучения весов функции оценки	86

Введение

Коридор — настольная логическая игра для двух или четырёх игроков, выпущенная в 1997 году компанией Gigamic. Она была создана Мирко Маркези на основе более ранней игры «Блокада». Игра получила заметное распространение среди настольных логических игр; в частности, она входила в подборку Games 100 журнала *Games* [3].

Общий вид партии показан на рисунке 1. На каждом ходу игрок выбирает одно из следующих действий: продвинуть свою фишку к противоположной стороне поля или поставить стену, которая усложнит маршрут соперника. Количество таких стен ограничено, а правила игры запрещают полностью блокировать путь любого игрока к его целевой стороне поля.

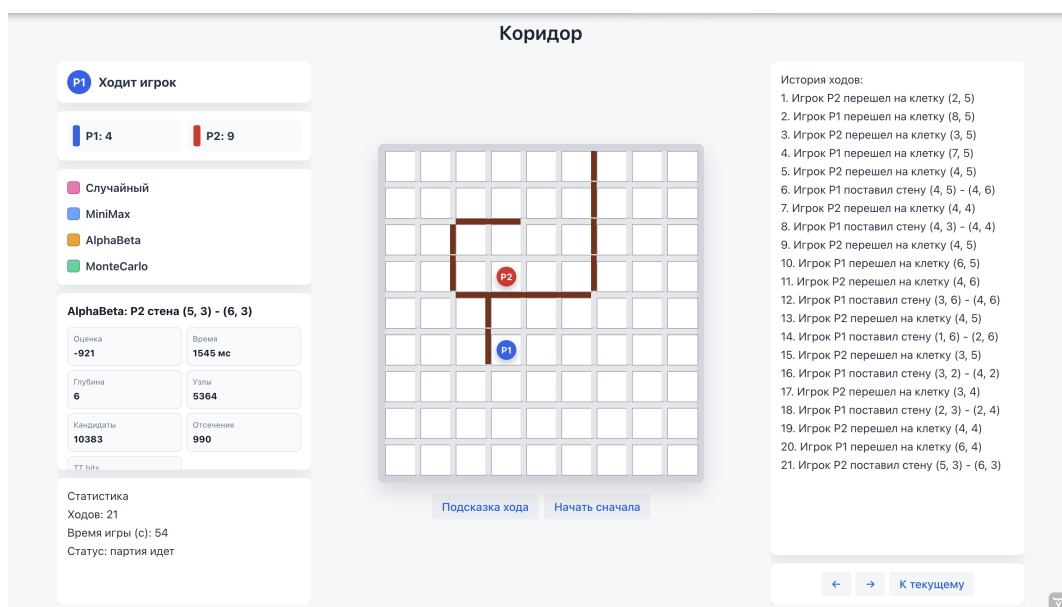


Рис. 1 — Пример партии в приложении «Коридор»

Целью выпускной квалификационной работы является разработка игрового приложения «Коридор», включающего реализацию правил игры, визуализацию игрового поля и модуль стратегического поведения, объединяющий четыре алгоритма программного соперника.

Для реализации цели работы были поставлены следующие задачи:

1. изучить правила игры «Коридор»,
2. реализовать алгоритм проверки корректности перемещения фишки,
3. реализовать алгоритм проверки корректности установки перегородки с проверкой достижимости целевых сторон поля,
4. реализовать алгоритм поиска кратчайшего пути до целевой стороны поля на основе обхода графа в ширину,

5. разработать функцию оценки игровой позиции с настраиваемыми весами и механизмом их коррекции по результатам партий,
6. реализовать четыре алгоритма программного соперника: случайный выбор хода, минимакс, альфа-бета отсечение и Монте-Карло,
7. провести сравнение алгоритмов в серии автоматических партий,
8. разработать интерфейс игрового приложения.

1. Игра «Коридор»

1.1. Необходимые определения

В формулировке правил игры и последующем описании алгоритмов встречается несколько понятий, которые нуждаются в пояснении.

Игровым полем называется квадратная доска размером $n \times n$ клеток, по которым перемещаются фишки игроков. В стандартном варианте игры $n = 9$, однако приложение также поддерживает поля 7×7 и 11×11 . Каждая из клеток нумеруется парой индексов (i, j) , где $0 \leq i, j \leq n - 1$.

Фишкой называется объект, который занимает одну клетку игрового поля и способен к перемещению на одну из смежных клеток.

Стеной называется элемент, который занимает пространство между клетками. Такая перегородка имеет длину две клетки и может быть расположена горизонтально, блокируя вертикальный проход между двумя парами клеток, или вертикально. Количество стен у каждого игрока ограничено: 8 для поля 7×7 , 10 для 9×9 , 12 для 11×11 .

Целевой линией игрока называется строка игрового поля, которая находится на противоположной стороне от его стартовой позиции. В начале партии игроки расположены на центрах противоположных сторон поля.

1.2. Описание игры

В начале партии фишки игроков расставляются на противоположных сторонах поля: первый игрок — в центре нижней строки, второй — в центре верхней строки. Игроки ходят по очереди, первенство определяется случайно. Цель каждого игрока — первым достичь любой клетки своей целевой линии.

За один ход игрок может выполнить только одно из двух действий:

1. Перемещение по игровому полю. Фишка игрока передвигается на смежную клетку, если такой маневр не заблокирован стеной. Если соседняя клетка занята фишкой соперника и за ней нет ни перегородки, ни границы поля, фишка игрока может перепрыгнуть через соперника. Если прямой прыжок невозможен, допускается прыжок на клетку слева или справа от той, куда изначально предполагался прыжок.
2. Установка стены. Она допускается, если: у игрока есть хотя бы одна перегородка в его запасе, она не пересекается с уже установленными стенами на игровом поле и после её установки у каждого из игроков остается хотя бы один маршрут к своей целевой линии.

Игра завершается, когда фишка одного из игроков достигает любой клетки целевой линии — этот игрок побеждает.

1.3. Модуль стратегического поведения соперника

Модуль стратегического поведения — это программный компонент, который отвечает за выбор хода соперника. Основываясь на полученных параметрах, а именно текущем состоянии игрового поля, количество оставшихся стен у игроков, модуль возвращает лучший по собственной оценке ход.

В процессе разработки к модулю стратегического поведения соперника были выдвинуты следующие требования:

1. *Корректность*: выбранный алгоритмом ход не должен противоречить правилам игры.
2. *Завершённость*: алгоритм должен завершаться за конечное время. Все реализованные в приложении алгоритмы работают с ограничением по времени: каждый из них получает лимит в миллисекундах и прекращает поиск по его истечении. Этот устанавливаемый лимит зависит от сложности игры и выбранного размера игрового поля.
3. *Единое взаимодействие с выбранным алгоритмом*: все алгоритмы реализуют единый программный интерфейс `Algorithm` с методом получения лучшего хода. Благодаря этому переключение алгоритмов или добавление новых не требует вносить какие-либо изменения в остальные компоненты игрового приложения.
4. *Настраиваемая сложность*: каждый из алгоритмов поддерживает три уровня сложности — EASY, MEDIUM и HARD. Выбранный уровень сложности влияет не только на ограничение по времени, но и на глубину поиска или число итераций.

В рамках данной работы в модуле поведения соперника реализовано четыре алгоритма, каждый из которых может быть выбран в качестве оппонента для игры против пользователя. Случайный алгоритм равновероятно выбирает один из множества допустимых ходов. Минимакс перебирает дерево игровых состояний на ограниченную глубину без каких-либо отсечений. Альфа-бета расширяет минимакс, добавляя в изначальный алгоритм отсечения безперспективных ветвей дерева. Метод Монте-Карло оценивает позиции через многократные случайные симуляции игры из каждой из них.

2. Алгоритмы

2.1. Общие функции и структура алгоритмов

Каждый из разработанных алгоритмов реализуют единый программный интерфейс `Algorithm`, содержащий как абстрактные методы, тело которых определяется выбранным алгоритмом, так и конкретные реализации общих вспомогательных функций. Благодаря этому исключается дублирование кода, упрощается добавление нового алгоритма в приложение, а также гарантируется единообразное поведение алгоритмов в одних и тех же ситуациях.

Генерация допустимых ходов

Метод `getMoves(board, playerId, wallsLeft)`, который применяется всеми алгоритмами в модуле стратегического поведения соперника, формирует полное множество допустимых ходов из текущего состояния из двух частей.

Первая часть — допустимые перемещения фишки, получаемые с помощью метода `board.getAvailableMoves(pos)`, который поочередно проверяет четыре направления перемещения с текущей клетки, а также обрабатывает прыжки через соперника, если такие возможны.

Вторая часть — допустимые установки стен. Если лимит стен игрока исчерпан, эта часть не дает никаких дополнительных вариантов ходов. В противном случае алгоритм формирует множество «кандидатных» клеток, ограничивая полный перебор стратегически значимой областью поля. Кандидатами являются клетки, которые расположены около кратчайшего пути соперника к его целевой строке поля. Для каждой такой клетки проверяются два возможных направления стены. Перегородка включается в список кандидатов, если она не имеет коллизий с уже расставленными перегородками на поле, и не отрезает ни одного из игроков от его целевой линии. Помимо прочего проверяется выполнение хотя бы одного условия:

1. стена увеличивает кратчайший путь соперника минимум на 2 хода
2. стена увеличивает кратчайший путь соперника не меньше, чем собственный путь алгоритма

Итоговый список возможных стен сортируется по убыванию функции `wallImpact`.

Поиск кратчайшего пути

Метод `calculateShortestPath(board, start, targetRow)` основан на обходе графа в ширину от текущей позиции игрового поля до ближайшей

клетки целевой строки.

Метод `getShortestPathCells` дополняет предыдущий восстановлением кратчайшего пути, поскольку при поиске такового хранит в истории посещенные клетки. Как уже описывалось ранее, он используется для поиска «кандидатных» клеток для установки стен.

Эндшпильные правила

Метод `findEndgameMove` вызывается в любом из алгоритмов в качестве первого шага перед запуском основного поиска лучшего хода. Если с текущей клетки поля фишку игрока можно переместить прямо его на целевую линию — этот ход немедленно возвращается без перебора других вариантов. Если такого «победного» хода нет, но соперник находится не более чем в двух ходах от победы, а лимит стен у алгоритма не исчерпан — ищется стена с максимальным значением `wallImpact`, чтобы замедлить игрока.

Предпочтение направления движения

С помощью метода `movementPreference` к оценке хода добавляется бонус или штраф в зависимости от направления перемещения и истории предыдущих ходов:

- ход в сторону целевой линии: +45 очков;
- ход назад: −120 очков;
- уменьшение длины пути до целевой линии на Δ клеток: $+35 \cdot \Delta$ очков;
- перемещение на позицию, которая встречалась в последних двух ходах: −500 очков;
- повтор более давнего перемещения: −180 очков.

Вся история перемещений хранится для каждого игрока отдельно и получается алгоритмом путем вызова `getRecentPositions`.

Функция оценки позиции

Функция оценки `evaluate()` является главным компонентом всех реализованных в игровом приложении алгоритмов. Благодаря ей каждому состоянию игры сопоставляется числовое значение, которое описывает его стратегическую выгоду для программного соперника. Если состояние игры подразумевает победу алгоритма или пользователя функция возвращает $+\text{WIN_SCORE} + d$ или $-\text{WIN_SCORE} - d$ соответственно, где $\text{WIN_SCORE} = 100\,000$, а d — текущая глубина поиска. С помощью этого параметра получается обеспечить предпочтение более близкой победы.

Для остальных состояний игры оценка вычисляется как взвешенная сумма следующих параметров:

$$F(s) = \sum_{k=1}^7 \lambda_k \cdot f_k(s),$$

где λ_k — веса, а $f_k(s)$ — значения параметров. Параметры и их описание перечислены в таблице 2.1.

Таблица 2.1

Параметры оценочной функции

k	Признак $f_k(s)$	Описание
1	$d_p - d_a$	Разность между длинами кратчайших путей алгоритма и пользователя
2	mob_a	Количество допустимых ходов фишкой алгоритма — его мобильность
3	$-\text{mob}_p$	Отрицательное количество допустимых ходов пользователя
4	$w_a - w_p$	Преимущество алгоритма по числу оставшихся стен
5	$\text{prog}_a - \text{prog}_p$	Разность продвижений к целевым линиям алгоритма и пользователя
6	$e(d_a)$	Эндшпильный бонус за близость алгоритма к победе над игроком
7	$-e(d_p)$	Эндшпильный штраф за близость пользователя к победе над алгоритмом

Продвижение prog_a равно количеству строк игрового поля, которые прошла фишка алгоритма от стартовой позиции до текущей. Эндшпильная функция $e(d)$ может принимать такие значения:

$$e(d) = \begin{cases} 10, & d \leq 1, \\ 4, & d = 2, \\ 1, & d = 3, \\ 0, & d > 3. \end{cases}$$

Веса λ_k , соответствующие описанным параметрам, по умолчанию заданы следующим образом (класс `EvaluationWeights`):

$$\lambda_1 = 65, \quad \lambda_2 = 4, \quad \lambda_3 = 3, \quad \lambda_4 = 18, \quad \lambda_5 = 6, \quad \lambda_6 = 165, \quad \lambda_7 = 175.$$

Коррекция весов

Игровое приложение поддерживает механизм автоматической коррекции параметров функции оценки, анализируя результаты сыгранных партий (класс `EvaluationWeightsStore`). После завершения каждой из партий, в которых участвовал программный соперник, вычисляется оценка полезности ходов, которые предпринимал алгоритм.

В течение игры для каждого хода алгоритма формируется объект `EvaluationLearning` который хранит вектор параметров до хода и после него. После завершения партии для каждого образца вычисляется дельта параметров оценки $\Delta f_k = f_k^{\text{after}} - f_k^{\text{before}}$ и суммируется в градиент с учётом знака:

$$g_k = \sum_{\text{ходы алгоритма}} \text{reward} \cdot \Delta f_k, \quad \text{reward} = \begin{cases} +1, & \text{алгоритм победил,} \\ -1, & \text{алгоритм проиграл.} \end{cases}$$

Если $g_k > 0$, увеличение веса k -го параметра чаще встречалось в ходах победителя, поэтому вес увеличивается. Если $g_k < 0$, этот признак чаще участвовал в принятии решений, приводящих к проигрышу, поэтому соответствующий ему вес уменьшается. Если $g_k = 0$, вес не изменяется. Для начала необходимо вычислить предварительное значение обновленного веса:

$$\lambda'_k = \lambda_k + \text{sign}(g_k) \cdot \Delta\lambda.$$

Затем оно проверяется согласно допустимому диапазону:

$$\lambda_k^{\text{new}} = \begin{cases} \lambda_k^{\min}, & \lambda'_k < \lambda_k^{\min}, \\ \lambda_k^{\max}, & \lambda'_k > \lambda_k^{\max}, \\ \lambda'_k, & \lambda_k^{\min} \leq \lambda'_k \leq \lambda_k^{\max}. \end{cases}$$

Здесь $\Delta\lambda = 1$ — это шаг обучения, а $\lambda_k^{\min}$ и $\lambda_k^{\max}$ — минимально и максимально допустимые значения веса. Такие ограничения были наложены на веса параметров, чтобы после серии обучающих партий с алгоритмом отдельный параметр не стал чрезмерно большим, отрицательным или слишком малым и не начал полностью подавлять остальные признаки функции оценки. Актуальные веса параметров регулярно сохраняются в файл `data/evaluation-weights.properties` и используются для функции оценки в следующем запуске приложения, что обеспечивает накопление опыта между сессиями. На момент написания работы механизм коррекции предпринял 15 обновлений весов на 1388 обучающих образцах, а текущие сохранённые веса составляют:

$$\lambda_1 = 120, \quad \lambda_2 = 7, \quad \lambda_3 = 12, \quad \lambda_4 = 8, \quad \lambda_5 = 25, \quad \lambda_6 = 186, \quad \lambda_7 = 170.$$

Сравнение изначально выбранных и обновленных весов показывает, что механизм увеличил приоритет разности путей ($\lambda_1 : 65 \rightarrow 120$), штрафа за количество возможных ходов соперника ($\lambda_3 : 3 \rightarrow 12$), продвижения к целевой строке игрового ($\lambda_5 : 6 \rightarrow 25$) и собственного эндшпильного преимущества ($\lambda_6 : 165 \rightarrow 186$), одновременно с этим уменьшив влияние преимущества по оставшимся стенам ($\lambda_4 : 18 \rightarrow 8$) на общую оценку состояния игры.

Функция wallImpact

Функция `wallImpact(board, move, playerId, size)` оценивает стратегическую ценность конкретной установки стены. Применяв ход на копии доски, она вычисляет изменения длин путей участников игры до их целевых строк:

$$\text{wallImpact} = (d_p^{\text{after}} - d_p^{\text{before}}) \cdot 100 - \max(0, d_a^{\text{after}} - d_a^{\text{before}}) \cdot 45,$$

то есть каждый шаг, увеличивающий путь соперника, оценивается в 100 единиц, а каждый шаг, удлиняющий собственный путь, штрафует 45 единицами. Стена считается стратегически ценной, если `wallImpact > 0`.

2.2. Случайный алгоритм

(RandomAlgorithm) является простейшим из возможных программных соперников. При каждом вызове метода `getMove` алгоритм первым делом применяет эндшпильное правило: если можно немедленно выиграть самому или срочно заблокировать путь для соперника, он делает это независимо от уровня сложности. Если такое правило не сработало, поведение определяется уровнем сложности.

На уровне EASY алгоритм с равной вероятностью выбирает ход из полного множества допустимых ходов.

На уровне сложности MEDIUM ходы сортируются по убыванию значения оценки хода, и случайно выбирается один из первой списка ходов, ограниченного первой его половиной.

На уровне HARD случайно выбирается уже один из трёх лучших ходов с помощью применения той же оценки.

Алгоритм 1: Случайный алгоритм

```
1  эндишпильный ход  $\leftarrow$  findEndgameMove( $s$ );
2  if эндишпильный ход найден then
3    |   return эндишпильный ход
4  end
5   $M \leftarrow$  getMoves( $s$ );
6  if уровень сложности = EASY then
7    |    $m^* \leftarrow$  случайный элемент  $M$ ;
8  else if уровень сложности = MEDIUM then
9    |   сортировать  $M$  по убыванию evaluateAfterMove;
10   |    $m^* \leftarrow$  случайный элемент из первой половины  $M$ ;
11 else
12   |   сортировать  $M$  по убыванию evaluateAfterMove;
13   |    $m^* \leftarrow$  случайный элемент из первых трёх элементов  $M$ ;
14 end
15 return  $m^*$ 
```

2.3. Минимакс

В MinimaxAlgorithm во время хода алгоритма выбирается ход с максимальной оценкой (максимизирующий узел); на ходах пользователя предполагается выбор хода с минимальной оценкой (минимизирующий узел).

Алгоритм итеративно увеличивает глубину анализа хода: последовательно запускает поиск с глубиной $d = 1, 2, \dots, d_{\max}$ до истечения лимита времени. На каждом этапе цикла в специальное поле сохраняется лучший выбранный ход, значение которого будет возвращено в качестве ответа. Благодаря этому, даже если лимит времени оказался исчерпан ранее, чем поиск на очередной глубине подошел к концу, всегда будет допустимый ответ, который можно будет использовать как ход алгоритма.

Максимальная глубина $d_{\max}$ и лимит времени T зависят от выбранных пользователем уровня сложности алгоритма и размера игрового поля:

Таблица 2.2

Параметры минимакса

Уровень	$d_{\max}$ (поле 9×9)	T , мс (поле 9×9)
EASY	3	700
MEDIUM	4	1800
HARD	5	3600

Чтобы алгоритм успевал обрабатывать большие значения глубины поиска за отведенное время, было введено ограничение на количество ходов для перебора на каждой итерации. После получения всего множества возможных вариантов хода из текущей позиции, эти ходы сортируются с помощью функции оценки, а

зачем уже отсортированный список обрезается первыми 16 вхождениями в него. Уже обработанные алгоритмом позиции и их оценки хранятся в таблице *cache*: в составе ключе соединены строковое представление доски, текущая глубина поиска, флаг максимизации или минимизации и количество оставшихся стен у обоих игроков. Общая схема работы алгоритма приведена в алгоритме 2.

Алгоритм 2: Минимакс с итеративным углублением

```

1  Функция minimax( $s, d, isMax$ ):
2      if в cache есть значение для ( $s, d, isMax$ ) then
3          | return значение из cache;
4      end
5      if terminal( $s$ ) then
6          | return терминальную оценку позиции;
7      end
8      if  $d = 0$  then
9          | return evaluate( $s$ );
10     end
11      $p \leftarrow$  игрок, которому принадлежит ход в узле  $s$ ;
12      $M \leftarrow$  лучшие 16 ходов из getMoves( $s, p$ );
13     if  $isMax$  then
14         |  $best \leftarrow -\infty$ ;
15         foreach  $m \in M$  do
16             |  $best \leftarrow \max(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{false}))$ ;
17         end
18     else
19         |  $best \leftarrow +\infty$ ;
20         foreach  $m \in M$  do
21             |  $best \leftarrow \min(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{true}))$ ;
22         end
23     end
24     сохранить  $best$  в cache для ( $s, d, isMax$ );
25     return  $best$ ;

26  $m^* \leftarrow$  первый допустимый ход;
27 for  $d \leftarrow 1$  to  $d_{\max}$  do
28     | if истёк лимит времени  $T$  then
29         | break
30     | end
31     |  $m^* \leftarrow \arg \max_{m \in M(s)} (\text{minimax}(\text{apply}(s, m), d - 1, \text{false}) +$ 
32         |  $\text{movementPreference}(m, s))$ ;
33 return  $m^*$ 

```

2.4. Альфа-бета отсечения

Алгоритм альфа-бета (AlphaBetaAlgorithm) является наиболее сложным из реализованных. Он расширяет минимакс тремя независимыми механизмами ускорения: отсечениями, сортировкой ходов и таблицей транспозиций.

Альфа-бета отсечение

Алгоритм поддерживает два параметра: α — наилучшая оценка, гарантированная максимизирующему игроку, и β — наилучшая оценка, гарантированная минимизирующему. Если в ходе перебора $\beta \leq \alpha$, оставшиеся ходы в данном узле можно не рассматривать: они не изменят итогового выбора. Это позволяет в лучшем случае сократить число рассматриваемых состояний до $O(b^{d/2})$.

Сортировка ходов

Эффективность отсечений критически зависит от порядка рассмотрения ходов: если лучший ход рассматривается первым, отсечения происходят как можно раньше. Функция `scoreMove` оценивает каждый ход до его применения по нескольким критериям:

- +20 000, если ход совпадает с записью из таблицы транспозиций (*лучший ход из предыдущего поиска*);
- +10 000 / +8 000, если ход является первым или вторым *killer-ходом* данного уровня глубины;
- вес из таблицы `goodMoves` — истории ходов, вызывавших отсечения в предыдущих узлах;
- значение функции оценки после применения хода.

После сортировки список ходов обрезаается до 36 кандидатов.

Таблица транспозиций

Таблица транспозиций (`transpositionTable`) хранит результаты уже вычисленных позиций. Ключ — строковое представление доски, флаг текущего игрока и количества перегородок. Каждая запись содержит оценку, глубину поиска, тип границы (точная оценка EXACT, нижняя граница LOWER или верхняя граница UPPER) и лучший ход для данной позиции. При попадании в таблицу алгоритм:

- если глубина записи $\geq$ текущей и тип EXACT — немедленно возвращает сохранённое значение;
- если тип LOWER — использует запись для ужесточения нижней границы α ;
- если тип UPPER — для ужесточения верхней границы β .

Затухающий поиск (Quiescence Search)

В позициях, где один из игроков находится не далее 2 ходов от победы (*шумные позиции*, `isNoisyPosition`), применяется расширенный поиск `quiescence`. Вместо немедленного возврата статической оценки алгоритм дополнительно рассматривает финальные ходы (перемещение фишки на целевую линию) и до 8 наиболее ценных установок перегородок с глубиной расширения 2. Это предотвращает ошибки горизонта — ситуации, когда статическая оценка не отражает угрозы, возникающей сразу после горизонта поиска.

Параметры алгоритма в зависимости от уровня сложности для поля 9×9 :

Таблица 2.3

Параметры альфа-бета алгоритма

Уровень	$d_{\max}$	T , мс
EASY	4	1300
MEDIUM	6	3600
HARD	8	7600

2.5. Монте-Карло

Алгоритм Монте-Карло (`MonteCarloAlgorithm`) строит статистику по многократным симуляциям от текущей позиции до конца партии [8]. В отличие от минимакса и альфа-бета, он не перебирает дерево равномерно до фиксированной глубины, а постепенно концентрирует вычислительный бюджет на наиболее перспективных ветвях. В реализации дополнительно используется эвристическая оценка позиции: она входит в UCT как небольшой нормализованный член и применяется при выборе ходов во время rollout. Структура алгоритма — дерево узлов, каждый из которых соответствует некоторому состоянию игры и хранит счётчики посещений n_v и побед w_v .

Основной цикл алгоритма состоит из четырёх фаз.

Выбор (Selection). Начиная с корня, алгоритм спускается по дереву, на каждом шаге выбирая дочерний узел с максимальным значением UCT:

$$\text{UCT}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} + h(v),$$

где $C = 1,2$ — константа исследования; $h(v)$ — нормализованная эвристическая оценка $0,15 \cdot \tanh(F(s_v)/500)$, добавляющая информацию о позиции без полного доминирования над статистикой. Если узел ни разу не посещался, ему присваивается приоритет $+\infty$.

Расширение (Expansion). Когда все допустимые ходы из узла уже рассматривались в дочерних узлах, выбранный узел помечается как полностью расширенный.

Алгоритм 3: Альфа-бета с таблицей транспозиций

```
1 Функция alphabeta( $s, d, \alpha, \beta, isMax$ ):
2   if запись ( $s, d$ )  $\in TT$  then
3     скорректировать  $\alpha, \beta$  по типу записи;
4     if  $\alpha \geq \beta$  then
5       return оценку из  $TT$ 
6     end
7   end
8   if terminal( $s$ ) then
9     return терминальная оценка
10  end
11  if  $d = 0$  then
12    if isNoisyPosition( $s$ ) then
13      return quiescence( $s, \alpha, \beta, isMax, 2$ )
14    else
15      return  $F(s)$ 
16    end
17  end
18   $M \leftarrow$  первые 36 ходов из getMoves( $s$ ), отсортированных по
    scoreMove;
19  best  $\leftarrow -\infty$  если isMax, иначе  $+\infty$ ;
20  foreach  $m \in M$  do
21     $v \leftarrow$  alphabeta(apply( $s, m$ ),  $d - 1, \alpha, \beta, \neg isMax$ );
22    обновить best,  $\alpha$  или  $\beta$ ;
23    if  $\beta \leq \alpha$  then
24      обновить killer-ходы и goodMoves;
25      break ; // отсечение
26    end
27  end
28  сохранить (best,  $d$ , тип) в  $TT$ ;
29  return best
```

Иначе из списка ещё не рассмотренных ходов (предварительно отсортированных по `scoreMove`) берётся первый, и создаётся новый дочерний узел.

Симуляция (Simulation). Из нового узла проводится случайная игра до терминального состояния или до исчерпания лимита шагов `rolloutDepth`. На каждом ходу симуляции применяется эвристика `epsilon-greedy`:

1. если есть ход, немедленно завершающий партию — он выбирается;
2. если соперник находится не далее 2 ходов от победы и у текущего игрока есть перегородки — выбирается перегородка с максимальным `wallImpact` (если её `wallImpact > 0`);
3. с вероятностью 0,22 — случайный ход из полного списка;
4. иначе — из случайной выборки 18 ходов выбирается один из четырёх лучших по `scoreRolloutMove`.

Обратное распространение (Backpropagation). Результат симуляции распространяется вверх: для каждого узла на пути от листа к корню инкрементируются n_v и w_v (если результат является победой корневого игрока).

После истечения лимита времени или достижения бюджета итераций выбирается ход с наибольшим значением $n_c + w_c/n_c$ среди дочерних узлов корня (*robustChild*), что отдаёт предпочтение хорошо исследованным ходам.

Параметры алгоритма для поля 9×9 :

Таблица 2.4

Параметры MCTS

Уровень	Бюджет итераций	T , мс	Глубина симуляции
EASY	520	950	$9 \cdot 4 = 36$
MEDIUM	1700	2900	$9 \cdot 7 = 63$
HARD	3400	6200	$9 \cdot 10 = 90$

Условие досрочного выхода (`bestWinRate > 0.97` при более чем 300 посещениях корня) позволяет экономить время в позициях, где один из ходов явно доминирует по статистике.

2.6. Сводные параметры сложности

Для удобства настройки экспериментов все уровни сложности сведены в таблицу 2.5. Значения приведены для стандартного поля 9×9 , поскольку именно оно используется в базовом сравнении алгоритмов. Для полей 7×7 и 11×11 параметры масштабируются в коде: на малом поле AlphaBeta может увеличивать глубину поиска, а на большом поле MiniMax и AlphaBeta уменьшают глубину либо увеличивают временной лимит, чтобы партия оставалась интерактивной.

Алгоритм 4: MCTS

```
1  $r \leftarrow$  корневой узел с состоянием  $s$ ;  
2 while не истёк лимит  $T$  и итераций  $< N$  do  
3    $v \leftarrow \text{select}(r)$ ; // спуск по UCT  
4   if  $\neg \text{terminal}(v.s)$  then  
5      $v \leftarrow \text{expand}(v)$ ; // новый дочерний узел  
6   end  
7    $\text{result} \leftarrow \text{simulate}(v, \text{rolloutDepth})$ ;  
8    $\text{backpropagate}(v, \text{result})$ ;  
9   if  $r.n > 300$  и  $\max_c(w_c/n_c) > 0.97$  then  
10    break  
11  end  
12 end  
13 return  $\arg \max_{c \in \text{children}(r)} (n_c + w_c/n_c)$ 
```

Таблица 2.5

Параметры уровней сложности алгоритмов для поля 9×9

Алгоритм	Уровень	Параметры
Random	EASY	Случайный выбор среди всех допустимых ходов
Random	MEDIUM	Случайный выбор из верхней половины ходов по функции оценки
Random	HARD	Случайный выбор из трёх лучших ходов по функции оценки
MiniMax	EASY	Глубина 3, лимит 700 мс, ширина перебора до 16 ходов
MiniMax	MEDIUM	Глубина 4, лимит 1800 мс, ширина перебора до 16 ходов
MiniMax	HARD	Глубина 5, лимит 3600 мс, ширина перебора до 16 ходов
AlphaBeta	EASY	Глубина 4, лимит 1300 мс, сортировка ходов, до 36 кандидатов
AlphaBeta	MEDIUM	Глубина 6, лимит 3600 мс, таблица транспозиций, killer-ходы
AlphaBeta	HARD	Глубина 8, лимит 7600 мс, quiescence search в шумных позициях
MCTS	EASY	До 520 итераций, лимит 950 мс, глубина rollout $9 \cdot 4 = 36$
MCTS	MEDIUM	До 1700 итераций, лимит 2900 мс, глубина rollout $9 \cdot 7 = 63$
MCTS	HARD	До 3400 итераций, лимит 6200 мс, глубина rollout $9 \cdot 10 = 90$

3. Игровое приложение

3.1. Техническая реализация

Игровое приложение написано на языке программирования Java версии 17 с использованием фреймворка Spring Boot версии 3.2.5, а также библиотеки Vaadin версии 24.4.2 [10], обеспечивающей отрисовку игрового поля и взаимодействие с пользователем через веб-интерфейс. Для сокращения шаблонного кода используется библиотека Lombok. Система сборки — Maven.

Архитектура приложения разделена на несколько независимых слоёв. Общая схема взаимодействия слоёв представлена на рисунке 2.



Рис. 2 — Общая архитектура приложения

Модель (`ru.ac.uniyar.model`) содержит классы, описывающие игровые объекты. Класс `Position` хранит координаты клетки (i, j) и предоставляет метод `move(rowDelta, colDelta)` для вычисления соседней позиции. Класс `BoardTile` описывает одну клетку поля четырьмя булевыми флагами доступности переходов. Класс `Board` хранит граф поля в виде `HashMap<Position, BoardTile>`, позиции фишек обоих игроков и реализует все операции с доской: размещение

перегородок (`placeWall`), получение доступных ходов (`getAvailableMoves`), обход для BFS (`getPathNeighbors`) и полное копирование (`copy`). Класс `Move` описывает ход: его тип (`MOVE_PLAYER` или `PLACE_WALL`), идентификатор игрока и позиции начала и конца. Класс `Game` хранит конфигурацию текущей партии: размер поля (`GameSize`), ссылки на игроков, номер текущего хода, время начала и флаг завершения.

Игроки (`ru.ac.uniyar.model.players`) реализуют иерархию через абстрактный класс `Player` с методом `getMove`. Класс `HumanPlayer` возвращает `null` (ход запрашивается через UI); класс `ComputerPlayer` делегирует вызов конкретному объекту `Algorithm`. Фабрика `ComputerPlayerFabric` создаёт нужную реализацию алгоритма по типу и уровню сложности.

Алгоритмы (`ru.ac.uniyar.model.algorithms`) описаны в главе 2. Все реализуют интерфейс `Algorithm`. Класс `AlgorithmReport` является неизменяемой записью с результатами последнего хода: выбранный ход, оценка, достигнутая глубина, число рассмотренных узлов, число отсечений, число попаданий в таблицу транспозиций, время в миллисекундах и текстовое пояснение. Класс `EvaluationFeatures` хранит вектор признаков состояния; `EvaluationWeights` — веса; `EvaluationWeightsStore` управляет загрузкой и сохранением весов.

Сервисный слой (`ru.ac.uniyar.service`) включает три компонента. `BoardFactory` создаёт начальное состояние доски заданного размера. `GameRules` проверяет корректность установки перегородки через BFS и конвертирует координаты рендера в игровые координаты. `GameProcessor` — аннотированный `@SessionScope` Spring-компонент, управляющий всем жизненным циклом партии: запуск (`startNewGame`), выполнение ходов (`makeMove`, `makeComputerMove`), режим воспроизведения (`replayPrevious`), сбор образцов для обучения весов и получение подсказок от всех четырёх алгоритмов. `BoardAnalyzer` формирует текстовые пояснения к ходам ИИ. `TournamentService` реализует автоматические серии партий для турнира и сравнения алгоритмов.

Интерфейсный слой (`ru.ac.uniyar.ui`) реализован через компоненты Vaadin. `StartPageController` отображает стартовый экран с выбором режима и параметров. `GamePageController` реализует основной экран игры: отрисовку доски в виде сетки Div-элементов, панель информации (`GameInfoPanel`), историю ходов (`GameHistoryPanel`), кнопки управления. `TournamentPageController` и `AlgorithmComparisonPageController` реализуют соответствующие режимы.

3.2. Взаимодействие с приложением

При запуске приложения пользователь попадает на стартовый экран, на котором настраиваются параметры предстоящей партии (рисунок 3).

Выбор размера поля. Доступны три варианта: малое поле 7×7 с 8 перегородками у каждого игрока, стандартное 9×9 с 10 перегородками, большое 11×11

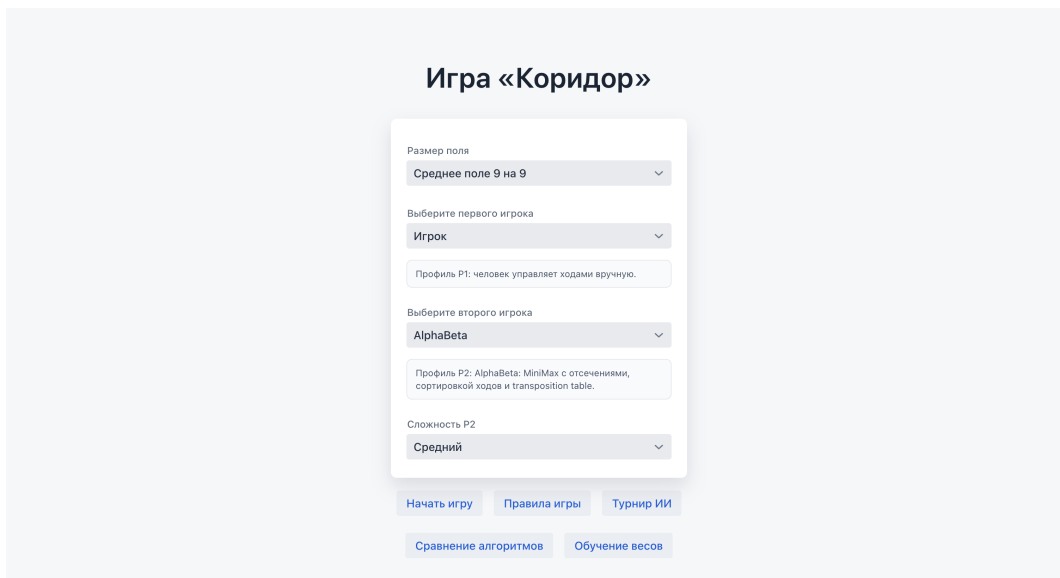


Рис. 3 — Стартовый экран выбора режима игры и алгоритмов

с 12 перегородками. Размер поля влияет на все параметры алгоритмов: лимиты времени и максимальные глубины масштабируются пропорционально.

Выбор первого игрока. Первым может управлять человек или один из четырёх алгоритмов ИИ. Если выбран ИИ, дополнительно настраивается его уровень сложности.

Выбор второго игрока. Второй игрок всегда управляется ИИ; выбирается алгоритм и уровень сложности. Карточки профилей отображают краткое описание выбранной стратегии.

На игровом экране пользователь видит несколько областей. Пример этого экрана приведён выше на рисунке 1.

Игровое поле отрисовывается в виде сетки: клетки (тайлы) чередуются с промежутками (щелями), в которых размещаются перегородки. Каждая клетка имеет размер 42–50 пикселей в зависимости от размера поля. Фишки отображаются цветными кружками (синий — P1, красный — P2). Допустимые для перемещения клетки подсвечиваются зелёным при наведении. При наведении на щель между клетками предварительно подсвечивается потенциальная перегородка.

Перемещение фишки выполняется кликом на целевую клетку. Если ход недопустим, появляется уведомление. Если ход совершается в режиме воспроизведения — он игнорируется.

Установка перегородки выполняется кликом на промежуток между клетками. `GameRules.extractWall` определяет по координатам рендера начальную и конечную клетки перегородки. Затем `GameRules.canPlaceWall` копирует доску, применяет перегородку и проверяет достижимость целевых линий обоих игроков через BFS.

Ход ИИ. В режиме «Игрок против ИИ» после хода пользователя автоматически запускается выбранный алгоритм. В режиме «ИИ против ИИ» каждый ход

требует нажатия кнопки «Сделать ход ИИ», что позволяет наблюдать за партией пошагово. Рядом с кнопкой отображается, чья очередь делать ход.

Подсказка. Кнопка «Подсказка хода» запускает все четыре алгоритма на текущей позиции и подсвечивает рекомендованные ими ходы разными цветами: синий (MiniMax), жёлтый (AlphaBeta), зелёный (MonteCarlo), розовый (Random). Если несколько алгоритмов рекомендуют один ход — клетка закрашивается градиентом. Пример отображения подсказок приведён на рисунке 4.

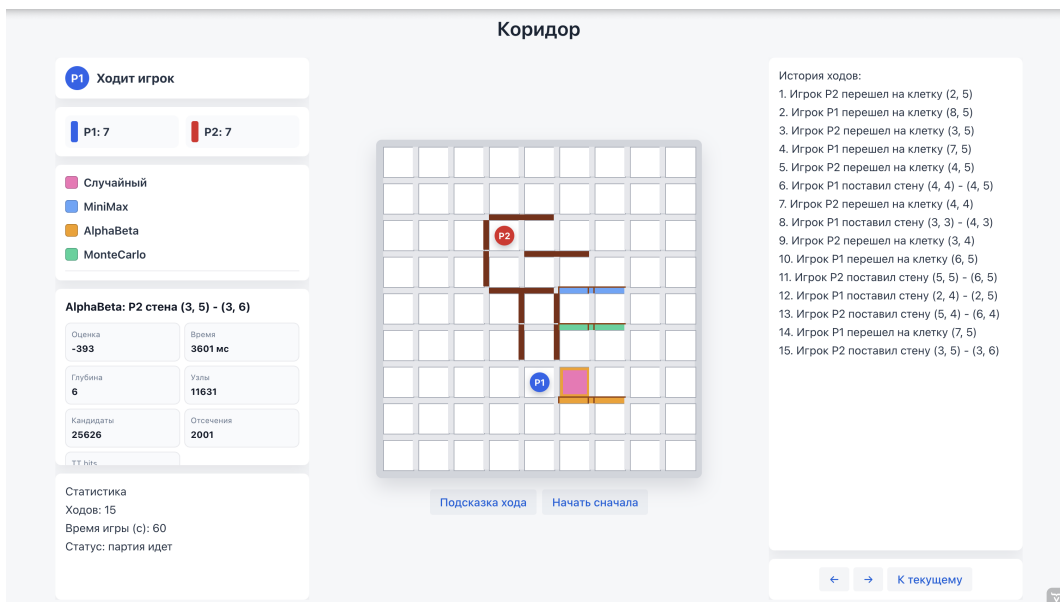


Рис. 4 — Отображение подсказок от алгоритмов на игровом поле

Панель информации отображает: чья очередь; количество оставшихся перегородок у каждого игрока; легенду цветов алгоритмов; отчёт о последнем ходе ИИ (оценка, время, глубина, узлы, отсечения, попадания в ТТ, текстовое пояснение); статистику партии (число ходов, время).

Режим воспроизведения. Кнопки «←», «→» и «К текущему» позволяют листать историю партии, просматривая состояние доски на любом из сыгранных ходов. История ходов дублируется текстовым списком в правой панели.

Турнирный режим позволяет задать два алгоритма, их уровни сложности, размер поля и число партий. Результаты выводятся в режиме реального времени: по завершении каждой партии обновляется счётчик побед. По завершении серии выводится сводная таблица. Пример работы режима показан на рисунке 5.

Режим сравнения алгоритмов автоматически проводит серию матчей между всеми попарными комбинациями четырёх алгоритмов (без зеркальных повторов) и выводит сводную таблицу с процентом побед, средним числом ходов, средним временем хода, средней глубиной поиска, числом узлов, отсечений и попаданий в ТТ. Интерфейс режима приведён на рисунке 6.

Режим обучения весов запускает серию self-play партий между двумя wybran-ными алгоритмами. По завершении каждой партии сервис обучения обновляет

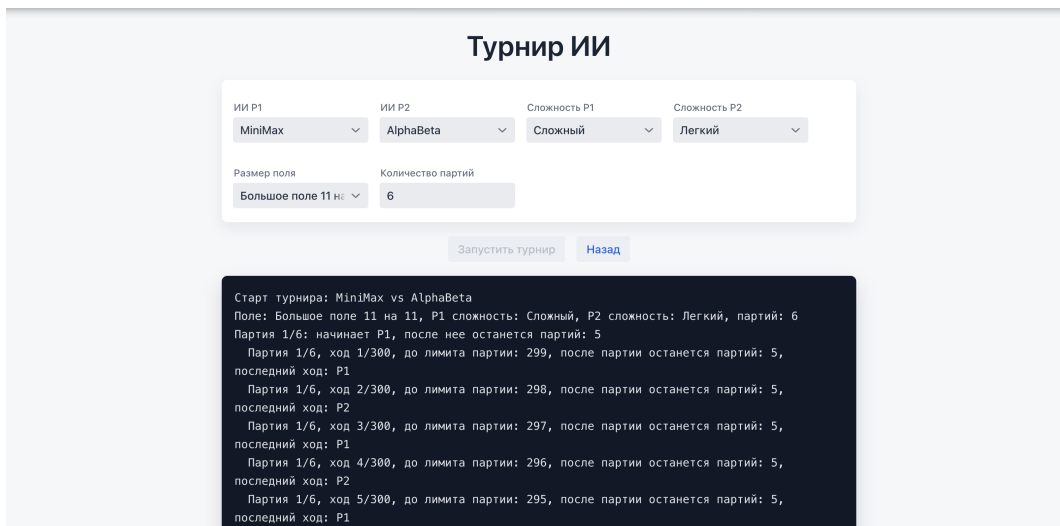


Рис. 5 — Экран турнирного режима ИИ

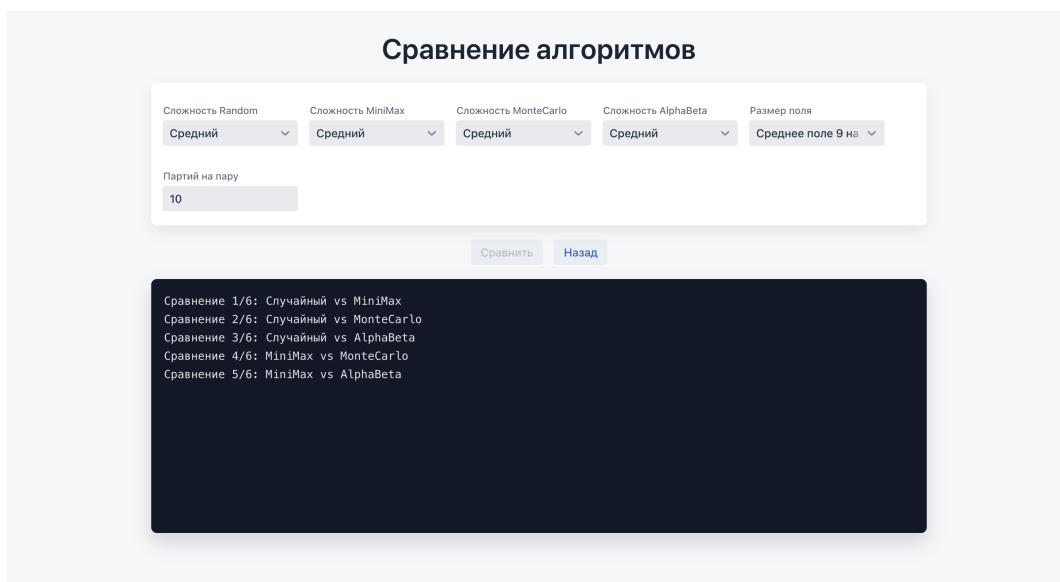


Рис. 6 — Экран сравнения алгоритмов

веса функции оценки, сохраняет их в файл и выводит ход эксперимента в лог. Пример логов обучения показан на рисунке 7.

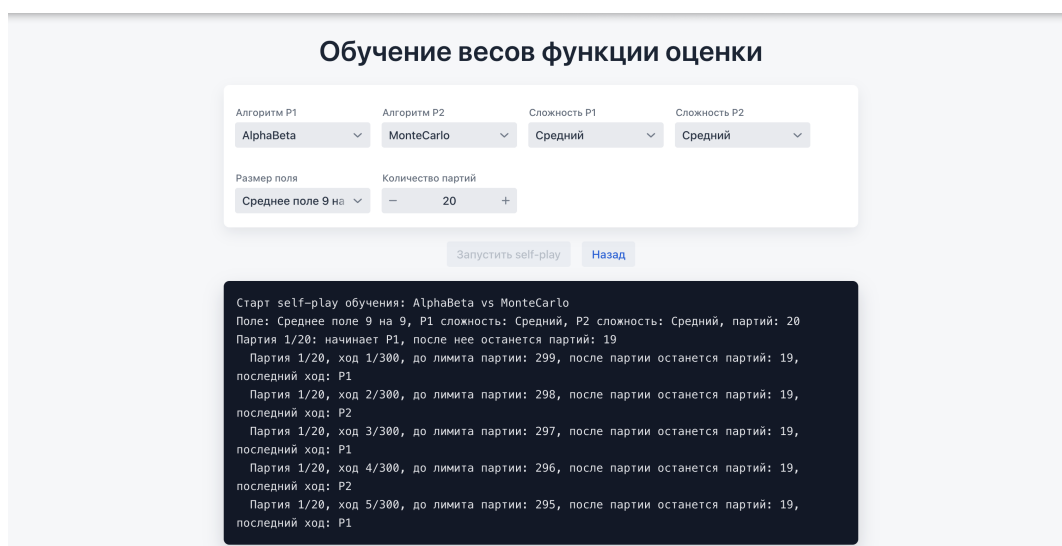


Рис. 7 — Экран self-play обучения весов функции оценки

4. Сравнение алгоритмов

4.1. Методика сравнения

Сравнение алгоритмов выполняется в отдельном режиме приложения «Сравнение алгоритмов». Цель режима — получить не единичный результат отдельной партии, а агрегированную оценку поведения стратегий на одинаковых условиях.

В эксперименте участвуют четыре алгоритма:

- RandomAlgorithm;
- MinimaxAlgorithm;
- AlphaBetaAlgorithm;
- MonteCarloAlgorithm.

Алгоритмы сравниваются попарно. Для исключения повторов рассматриваются только неупорядоченные пары: если матч MiniMax против Random уже проведён, обратная пара Random против MiniMax отдельно не запускается. При четырёх алгоритмах количество различных пар равно:

$$C_4^2 = \frac{4 \cdot 3}{2} = 6.$$

Внутри каждой пары проводится заданное пользователем число партий. Для компенсации преимущества первого хода роли игроков чередуются: часть партий алгоритм проводит за первого игрока, часть — за второго. Размер поля и уровень сложности каждого алгоритма задаются перед запуском эксперимента.

Важно, что в режиме сравнения алгоритмов обучение весов функции оценки не выполняется. Это сделано намеренно: если во время эксперимента функция оценки будет изменяться, более поздние партии будут проходить уже при других условиях, и сравнение потеряет воспроизводимость. Поэтому веса считаются фиксированными на протяжении всего запуска сравнения.

4.2. Собираемые метрики

Для анализа недостаточно знать только победителя. Два алгоритма могут иметь близкую долю побед, но существенно отличаться по времени работы, глубине поиска или количеству просмотренных состояний. Поэтому приложение собирает несколько групп метрик.

Метрики глубины, узлов, отсечений и попаданий в таблицу транспозиций особенно важны для сравнения MinimaxAlgorithm и AlphaBetaAlgorithm. Оба алгоритма основаны на дереве игры, но альфа-бета должен просматривать

Метрики сравнения алгоритмов

Метрика	Смысл
Количество партий	Общее число завершённых партий с участием алгоритма
Победы, поражения, ничьи	Основная игровая результативность алгоритма
Win rate	Доля побед алгоритма среди всех сыгранных им партий
Среднее число ходов	Косвенная характеристика стиля игры и сложности партий
Среднее время хода	Практическая вычислительная стоимость алгоритма
Средняя глубина	Насколько глубоко алгоритм успевает просмотреть дерево
Среднее число узлов	Объём реально просмотренного дерева поиска
Среднее число отсечений	Эффективность alpha-beta оптимизации
Попадания в ТТ	Эффективность таблицы транспозиций

меньше узлов при большей достижимой глубине. Для MonteCarloAlgorithm ключевыми являются число итераций и время хода, поскольку качество решения определяется количеством проведённых симуляций.

4.3. План эксперимента

Базовый эксперимент проводится на стандартном поле 9×9 . Для каждого алгоритма устанавливается уровень сложности MEDIUM. Каждая пара алгоритмов играет по 10 партий. Тогда суммарное число партий равно:

$$6 \cdot 10 = 60.$$

После базового эксперимента целесообразно провести дополнительный запуск на уровне HARD, поскольку именно на высоком уровне сложности сильнее проявляются различия между переборными алгоритмами. AlphaBeta получает возможность искать глубже за счёт отсечений, тогда как MiniMax ограничивается меньшей глубиной из-за экспоненциального роста дерева.

4.4. Форма представления результатов

Итоговая статистика может быть представлена в двух формах. Первая форма — попарная таблица побед, показывающая, как конкретные алгоритмы играют друг против друга.

Таблица 4.2

Попарные результаты сравнения алгоритмов

Алгоритм P1	Алгоритм P2	Игры	Победы P1	Победы P2	Ничьи	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч. / ТТ
Случайный	MiniMax	10	0	8	2	66,50	151,2	2,46	1088,5	0,0 / 0,0
Случайный	MonteCarlo	10	0	10	0	142,50	1051,7	31,67	140,8	0,0 / 0,0
Случайный	AlphaBeta	10	0	10	0	73,50	412,4	3,41	1307,5	249,2 / 198,9
MiniMax	MonteCarlo	10	4	6	0	98,30	1360,1	33,00	947,1	0,0 / 0,0
MiniMax	AlphaBeta	10	1	1	8	41,80	1215,3	4,85	4652,7	503,6 / 371,0
MonteCarlo	AlphaBeta	10	1	6	3	79,90	1839,7	34,28	1504,6	266,5 / 217,4

Вторая форма — агрегированная таблица по каждому алгоритму. Она удобнее для общего вывода, поскольку показывает не только победы, но и вычислительные затраты.

Таблица 4.3

Сводная статистика алгоритмов

Алгоритм	Партий	Победы	Поражения	Ничьи	Winrate	Очки/партия	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч. / ТТ
Случайный	30	0	28	2	0,00%	0,03	94,17	538,4	12,51	845,6	83,1 / 66,3
MiniMax	30	13	7	10	43,33%	0,60	68,87	908,9	13,44	2229,4	167,9 / 123,7
MonteCarlo	30	17	10	3	56,67%	0,62	106,90	1417,2	32,98	864,2	88,8 / 72,5
AlphaBeta	30	17	2	11	56,67%	0,75	65,07	1155,8	14,18	2488,3	339,8 / 262,4

Для AlphaBeta дополнительно анализируются среднее число отсечений и попаданий в таблицу транспозиций. Эти показатели позволяют подтвердить, что улучшение относительно MiniMax связано не только с увеличенным лимитом времени, но и с реальным сокращением числа просматриваемых ветвей.

4.5. Анализ результатов

Фактические результаты подтверждают, что случайный алгоритм является базовой линией: за 30 партий он не одержал ни одной победы и набрал только две ничьи. Это ожидаемо, поскольку даже при использовании функции оценки он не строит дерево последствий и не анализирует ответы соперника.

MiniMax заметно сильнее случайного алгоритма: он выиграл 13 партий из 30 и набрал 0,60 очка за партию. При этом его слабое место сохраняется — экспоненциальный рост числа состояний. Даже при ограничении ширины до 16 ходов глубина 5 даёт до $16^5 \approx 1,05 \cdot 10^6$ узлов в худшем случае, поэтому алгоритм вынужден жёстко ограничивать глубину и ширину перебора.

AlphaBeta показал лучший интегральный результат: 17 побед, только 2 поражения и 0,75 очка за партию. При равном winrate с MonteCarlo он имеет меньше поражений и больше ничьих, что говорит о более устойчивом поведении в сложных позициях. Среднее число отсечений 339,8 и 262,4 попадания в таблицу транспозиций за ход подтверждают, что преимущество алгоритма связано не только с перебором, но и с эффективным сокращением дерева поиска.

MonteCarlo также показал высокий результат — 17 побед из 30 и winrate 56,67%. Его партии в среднем длиннее, чем у AlphaBeta, а среднее время хода

выше. Это отражает природу метода: качество решения зависит от числа симуляций и устойчивости rollout-стратегии. В игре «Коридор» это особенно важно из-за большого числа возможных стен: если бюджет итераций мал, статистика по отдельным ходам может быть недостаточно устойчивой.

4.6. Ограничения эксперимента

При интерпретации результатов необходимо учитывать несколько ограничений.

Во-первых, алгоритмы используют разные типы вычислительного бюджета: MiniMax и AlphaBeta ограничены глубиной и временем, а MCTS — временем, числом итераций и глубиной симуляции. Поэтому сравнение не является сравнением «одинаковых» алгоритмов при одинаковой глубине, а является сравнением реализованных игровых стратегий в условиях приложения.

Во-вторых, результат зависит от текущих весов функции оценки. Для воспроизводимости в тексте работы необходимо указать значения весов из файла `data/evaluation-weights.properties`, использованные перед запуском эксперимента.

В-третьих, выбор числа партий влияет на статистическую устойчивость результатов. Серия из 60 партий подходит для первичного сравнения, но для более надёжного вывода желательно увеличить число партий в каждой паре.

Заключение

В ходе выполнения выпускной квалификационной работы была исследована настольная логическая игра «Коридор» и построена её графовая модель. Реализованы алгоритмы проверки корректности перемещения фишки с учётом всех правил прыжков через соперника и проверки корректности установки перегородки через обход графа в ширину. Реализован алгоритм поиска кратчайшего пути до целевой линии на основе BFS с восстановлением пути.

Разработана многофакторная функция оценки позиции, учитывающая семь признаков: разность длин путей, мобильность фишек, преимущество в перегородках, продвижение к цели и эндшпильные бонусы. Реализован механизм коррекции весов функции оценки по результатам завершённых партий с сохранением в файл конфигурации между сессиями.

Реализованы четыре алгоритма программного соперника с тремя уровнями сложности каждый. Случайный алгоритм служит базовой линией. Минимакс с кэшированием и ограничением ширины перебора обеспечивает разумный баланс качества и скорости. Минимакс с альфа-бета отсечением, сортировкой ходов, таблицей транспозиций с типами границ и затухающим поиском в шумных позициях является наиболее сильным из переборных алгоритмов. Метод Монте-Карло с UCT, эвристическим выбором ходов в симуляциях и условием досрочного останова обеспечивает альтернативный подход, основанный на статистике симуляций.

Разработано игровое приложение с режимами «Игрок против ИИ», «ИИ против ИИ», режимом подсказок одновременно от всех четырёх алгоритмов, режимом воспроизведения партии, турнирным режимом и режимом сравнения алгоритмов. После проведения экспериментального запуска таблицы сравнения алгоритмов должны быть заполнены фактическими результатами.

Список литературы

- [1] Gigamic. Правила игры Коридор [Электронный ресурс]. — Режим доступа: <https://en.gigamic.com/modern-classics/107-quoridor.html> (дата обращения: 29.04.2026).
- [2] BoardGameGeek. Коридор [Электронный ресурс]. — Режим доступа: <https://boardgamegeek.com/boardgame/624/quoridor> (дата обращения: 29.04.2026).
- [3] BoardGameGeek. Games 100 [Electronic resource]. — Режим доступа: https://boardgamegeek.com/wiki/page/Games_100 (online; accessed: 02.04.2026).
- [4] Games, Hobby. Коридор. Правила игры [Электронный ресурс]. — Режим доступа: https://hobbygames.ru/download/rules/Quoridor_Rules.pdf (дата обращения: 03.04.2026).
- [5] Mertens, J. A Koridor-playing agent [Text] / J. Mertens // Technical report. — 2006.
- [6] Hart, P. E. A Formal Basis for the Heuristic Determination of Minimum Cost Paths [Text] / P. E. Hart, N. J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. — 1968. — Vol. 4, no. 2. — P. 100–107.
- [7] Russell, S. Artificial Intelligence: A Modern Approach [Text] / S. Russell, P. Norvig. — Hoboken : Pearson, 2021.
- [8] Coulom, R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search [Text] / R. Coulom // Computers and Games. — 2006. — P. 72–83.
- [9] Browne, C. Connection Games: Variations on a Theme [Text] / C. Browne. — Wellesley : A K Peters, 2009.
- [10] Vaadin. Vaadin Flow Documentation [Electronic resource]. — Режим доступа: <https://vaadin.com/docs> (online; accessed: 29.04.2026).

Реализация интерфейса алгоритма

Листинг А.1 — Интерфейс Algorithm

```

1  public interface Algorithm {
2      int WIN_SCORE = 100_000;
3
4      ComputerAlgorithmType getType();
5      Move getMove(Board board, ComputerPlayerHardnessLevel
        hardnessLevel, int playerId, int wallsLeft1, int
        wallsLeft2);
6
7      default AlgorithmReport getLastReport() {
8          return null;
9      }
10
11     default void setRecentPositions(List<Position>
        recentPositions) {
12     }
13
14     default int movementPreference(Move move, Board board, int
        playerId, int size, List<Position> recentPositions) {
15         if (move.getMoveType() != MoveType.MOVE_PLAYER || move.
            getPlayerId() != playerId) {
16             return 0;
17         }
18
19         int score = 0;
20         Position current = getCurrentPosition(board, playerId);
21         Position target = move.getEndPosition();
22         int direction = playerId == 1 ? -1 : 1;
23         int rowDelta = target.row() - current.row();
24
25         if (rowDelta == direction) {
26             score += 45;
27         } else if (rowDelta == -direction) {
28             score -= 120;
29         }

```

```

30
31     for (int index = 0; index < recentPositions.size(); ++
        index) {
32         Position recent = recentPositions.get(index);
33         if (recent.equals(current)) {
34             continue;
35         }
36         if (target.equals(recent)) {
37             score -= index <= 1 ? 500 : 180;
38         }
39     }
40
41     int before = Math.abs(current.row() - getTargetRow(
        playerId, size));
42     int after = Math.abs(target.row() - getTargetRow(
        playerId, size));
43     score += (before - after) * 35;
44     return score;
45 }
46
47 default List<Move> getMoves(Board board, int playerId, int
    wallsLeft) {
48     List<Move> moves = new ArrayList<>();
49     Position pos = getCurrentPosition(board, playerId);
50
51     for (Position next : board.getAvailableMoves(pos)) {
52         moves.add(Move.movePlayer(playerId, next));
53     }
54
55     if (wallsLeft <= 0) {
56         return moves;
57     }
58
59     int size = (int) Math.sqrt(board.getTiles().size());
60     Set<String> used = new HashSet<>();
61     List<Position> enemyPath = getShortestPathCells(board,
        getCurrentPosition(board, 3 - playerId),
62         getTargetRow(3 - playerId, size));
63     List<Position> myPath = getShortestPathCells(board,
        getCurrentPosition(board, playerId),

```

```

64         getTargetRow(playerId, size));
65     List<Position> candidateCells = new ArrayList<>();
66     List<Position> enemyFront = enemyPath.subList(0, Math.
        min(12, enemyPath.size()));
67     List<Position> myFront = myPath.subList(0, Math.min(7,
        myPath.size()));
68     candidateCells.addAll(enemyFront);
69     candidateCells.addAll(myFront);
70     addNeighborhood(candidateCells, getCurrentPosition(
        board, 3 - playerId), size);
71     for (Position cell : enemyFront.subList(0, Math.min(5,
        enemyFront.size())) {
72         addNeighborhood(candidateCells, cell, size, 1);
73     }
74
75     int beforeEnemy = calculateShortestPath(board,
        getCurrentPosition(board, 3 - playerId),
        getTargetRow(3 - playerId, size));
76     int beforeMine = calculateShortestPath(board,
        getCurrentPosition(board, playerId), getTargetRow(
        playerId, size));
77     List<Move> wallMoves = new ArrayList<>();
78
79     for (Position cell : candidateCells) {
80         int i = cell.row();
81         int j = cell.col();
82
83         if (i < size - 1) {
84             Position start = new Position(i, j);
85             Position end = new Position(i + 1, j);
86             String wallKey = start + "-" + end;
87
88             if (used.add(wallKey) && isWallPlaceable(board,
                start, end) && isValidWall(board, start,
                end)) {
89                 if (isStrategicWall(board, start, end,
                    playerId, size, beforeEnemy, beforeMine)
                    ) {
90                     wallMoves.add(Move.placeWall(playerId,
                        start, end));

```

```

91         }
92     }
93 }
94
95     if (j < size - 1) {
96         Position start = new Position(i, j);
97         Position end = new Position(i, j + 1);
98         String wallKey = start + "-" + end;
99
100         if (used.add(wallKey) && isWallPlaceable(board,
101             start, end) && isValidWall(board, start,
102             end)) {
103             if (isStrategicWall(board, start, end,
104                 playerId, size, beforeEnemy, beforeMine)
105                 ) {
106                 wallMoves.add(Move.placeWall(playerId,
107                     start, end));
108             }
109         }
110     }
111     wallMoves.sort((a, b) -> Integer.compare(
112         wallImpact(board, b, playerId, size),
113         wallImpact(board, a, playerId, size)
114     ));
115     moves.addAll(wallMoves);
116     return moves;
117 }
118
119 default void addNeighborhood(List<Position> cells, Position
120     center, int size) {
121     addNeighborhood(cells, center, size, 2);
122 }
123
124 default void addNeighborhood(List<Position> cells, Position
125     center, int size, int radius) {
126     for (int di = -radius; di <= radius; ++di) {
127         for (int dj = -radius; dj <= radius; ++dj) {
128             int nextRow = center.row() + di;
129             int nextCol = center.col() + dj;

```

```

124         if (nextRow >= 0 && nextCol >= 0 && nextRow <
            size && nextCol < size) {
125             cells.add(new Position(nextRow, nextCol));
126         }
127     }
128 }
129 }
130
131 default boolean isStrategicWall(Board board, Position start
    , Position end, int playerId, int size,
132         int beforeEnemy, int
            beforeMine) {
133     Board copy = board.copy();
134     copy.placeWall(start, end);
135
136     int afterEnemy = calculateShortestPath(copy,
        getCurrentPosition(copy, 3 - playerId), getTargetRow(
        3 - playerId, size));
137     int afterMine = calculateShortestPath(copy,
        getCurrentPosition(copy, playerId), getTargetRow(
        playerId, size));
138
139     int enemyGain = afterEnemy - beforeEnemy;
140     int myCost = Math.max(0, afterMine - beforeMine);
141
142     if (enemyGain >= 2) {
143         return true;
144     }
145     if (beforeEnemy <= 3 && enemyGain > 0) {
146         return true;
147     }
148     return enemyGain > 0 && enemyGain >= myCost;
149 }
150
151 default boolean isWallPlaceable(Board board, Position start
    , Position end) {
152     try {
153         Board copy = board.copy();
154         copy.placeWall(start, end);
155         return true;

```

```

156         } catch (Exception e) {
157             return false;
158         }
159     }
160
161     default List<Position> getShortestPathCells(Board board,
162         Position start, int targetRow) {
163         Map<Position, Position> parent = new HashMap<>();
164         Queue<Position> queue = new ArrayDeque<>();
165         Set<Position> visited = new HashSet<>();
166
167         queue.add(start);
168         visited.add(start);
169
170         Position end = null;
171         while (!queue.isEmpty()) {
172             Position curr = queue.poll();
173
174             if (curr.row() == targetRow) {
175                 end = curr;
176                 break;
177             }
178
179             for (Position next : board.getPathNeighbors(curr))
180             {
181                 if (visited.add(next)) {
182                     parent.put(next, curr);
183                     queue.add(next);
184                 }
185             }
186
187             List<Position> path = new ArrayList<>();
188             while (end != null) {
189                 path.add(end);
190                 end = parent.get(end);
191             }
192             Collections.reverse(path);
193             return path;
194         }

```

```

194
195     default boolean isValidWall(Board board, Position start,
196         Position end) {
197         try {
198             Board copy = board.copy();
199             copy.placeWall(start, end);
200
201             int size = (int) Math.sqrt(board.getTiles().size())
202                 ;
203
204             return hasPath(copy, copy.getPositionOfPlayer1(),
205                 0) && hasPath(copy, copy.getPositionOfPlayer2(),
206                 size - 1);
207         } catch (Exception e) {
208             return false;
209         }
210     }
211
212     private boolean hasPath(Board board, Position start, int
213         targetRow) {
214         Set<Position> visited = new HashSet<>();
215         Queue<Position> queue = new ArrayDeque<>();
216
217         queue.add(start);
218         visited.add(start);
219
220         while (!queue.isEmpty()) {
221             Position curr = queue.poll();
222
223             if (curr.row() == targetRow) {
224                 return true;
225             }
226
227             for (Position next : board.getPathNeighbors(curr))
228                 {
229                 if (visited.add(next)) {
230                     queue.add(next);
231                 }
232             }
233         }
234     }

```

```

228         return false;
229     }
230
231     default void applyMove(Board board, Move move) {
232         if (move.getMoveType() == MoveType.MOVE_PLAYER) {
233             if (move.getPlayerId() == 1) {
234                 board.setPositionOfPlayer1(move.getEndPosition
235                     ());
236             } else {
237                 board.setPositionOfPlayer2(move.getEndPosition
238                     ());
239             }
240         } else {
241             board.placeWall(move.getStartPosition(), move.
242                 getEndPosition());
243         }
244     }
245
246     default Position getCurrentPosition(Board board, int
247         playerId) {
248         return playerId == 1 ? board.getPositionOfPlayer1() :
249             board.getPositionOfPlayer2();
250     }
251
252     default int getTargetRow(int playerId, int size) {
253         return playerId == 1 ? 0 : size - 1;
254     }
255
256     default boolean isWin(Board board, int playerId, int size)
257     {
258         return getCurrentPosition(board, playerId).row() ==
259             getTargetRow(playerId, size);
260     }
261
262     default Integer terminalScore(Board board, int rootPlayerId
263         , int size, int depth) {
264         if (isWin(board, rootPlayerId, size)) {
265             return WIN_SCORE + depth;
266         }
267         if (isWin(board, 3 - rootPlayerId, size)) {

```



```

260         return -WIN_SCORE - depth;
261     }
262     return null;
263 }
264
265 default Move findEndgameMove(Board board, int playerId, int
    wallsLeft) {
266     int size = (int) Math.sqrt(board.getTiles().size());
267     Position current = getCurrentPosition(board, playerId);
268     for (Position next : board.getAvailableMoves(current))
269     {
270         if (next.row() == getTargetRow(playerId, size)) {
271             return Move.movePlayer(playerId, next);
272         }
273     }
274
275     if (wallsLeft <= 0) {
276         return null;
277     }
278
279     int enemy = 3 - playerId;
280     int enemyDistance = calculateShortestPath(board,
        getCurrentPosition(board, enemy), getTargetRow(enemy
        , size));
281
282     if (enemyDistance > 2) {
283         return null;
284     }
285
286     Move bestWall = null;
287     int bestImpact = 0;
288     for (Move move : getMoves(board, playerId, wallsLeft))
289     {
290         if (move.getMoveType() != MoveType.PLACE_WALL) {
291             continue;
292         }
293         int impact = wallImpact(board, move, playerId, size
        );
294         if (impact > bestImpact) {
295             bestImpact = impact;
296             bestWall = move;
297         }
298     }
299     return bestWall;
300 }

```

```

294         }
295     }
296     return bestImpact > 0 ? bestWall : null;
297 }
298
299 default boolean isNoisyPosition(Board board, int playerId,
300     int size) {
301     int myDist = calculateShortestPath(board,
302         getCurrentPosition(board, playerId), getTargetRow(
303             playerId, size));
304     int enemyDist = calculateShortestPath(board,
305         getCurrentPosition(board, 3 - playerId),
306         getTargetRow(3 - playerId, size));
307     return myDist <= 2 || enemyDist <= 2;
308 }
309
310 default List<Move> getQuiescenceMoves(Board board, int
311     currentPlayer, int wallsLeft, int ignoredRootPlayerId,
312     int size) {
313     List<Move> moves = new ArrayList<>();
314     Position current = getCurrentPosition(board,
315         currentPlayer);
316     for (Position next : board.getAvailableMoves(current))
317     {
318         if (next.row() == getTargetRow(currentPlayer, size)
319             ) {
320             moves.add(Move.movePlayer(currentPlayer, next))
321                 ;
322         }
323     }
324
325     if (wallsLeft > 0) {
326         List<Move> tacticalWalls = getMoves(board,
327             currentPlayer, wallsLeft).stream()
328             .filter(move -> move.getMoveType() ==
329                 MoveType.PLACE_WALL)
330             .sorted((a, b) -> Integer.compare(
331                 wallImpact(board, b, currentPlayer,
332                     size),
333                 wallImpact(board, a, currentPlayer,

```

```

size)
320         ))
321         .limit(8)
322         .toList();
323     moves.addAll(tacticalWalls);
324 }
325     return moves;
326 }
327
328     default int wallImpact(Board board, Move move, int playerId
, int size) {
329         int enemy = 3 - playerId;
330         int beforeEnemy = calculateShortestPath(board,
            getCurrentPosition(board, enemy), getTargetRow(enemy
, size));
331         int beforeMine = calculateShortestPath(board,
            getCurrentPosition(board, playerId), getTargetRow(
playerId, size));
332         Board copy = board.copy();
333         applyMove(copy, move);
334         int afterEnemy = calculateShortestPath(copy,
            getCurrentPosition(copy, enemy), getTargetRow(enemy,
size));
335         int afterMine = calculateShortestPath(copy,
            getCurrentPosition(copy, playerId), getTargetRow(
playerId, size));
336         return (afterEnemy - beforeEnemy) * 100 - Math.max(0,
            afterMine - beforeMine) * 45;
337     }
338
339     default int evaluate(Board board, int playerId, int size,
int wallsLeft1, int wallsLeft2) {
340         Integer terminal = terminalScore(board, playerId, size,
0);
341         if (terminal != null) {
342             return terminal;
343         }
344
345         return evaluationFeatures(board, playerId, size,
            wallsLeft1, wallsLeft2)

```

```

346         .score(EvaluationWeightsStore.current());
347     }
348
349     default EvaluationFeatures evaluationFeatures(Board board,
        int playerId, int size, int wallsLeft1, int wallsLeft2)
    {
350         int myDist = calculateShortestPath(board,
            getCurrentPosition(board, playerId), getTargetRow(
                playerId, size));
351         int enemyDist = calculateShortestPath(board,
            getCurrentPosition(board, 3 - playerId),
            getTargetRow(3 - playerId, size));
352
353         int myWalls = playerId == 1 ? wallsLeft1 : wallsLeft2;
354         int enemyWalls = playerId == 1 ? wallsLeft2 :
            wallsLeft1;
355
356         int myRow = getCurrentPosition(board, playerId).row();
357         int enemyRow = getCurrentPosition(board, 3 - playerId).
            row();
358         int myProgress = playerId == 1 ? size - 1 - myRow :
            myRow;
359         int enemyProgress = playerId == 1 ? enemyRow : size - 1
            - enemyRow;
360
361         return new EvaluationFeatures(
362             enemyDist - myDist,
363             board.getAvailableMoves(getCurrentPosition(
                board, playerId)).size(),
364             -board.getAvailableMoves(getCurrentPosition(
                board, 3 - playerId)).size(),
365             myWalls - enemyWalls,
366             myProgress - enemyProgress,
367             endgameFeature(myDist),
368             -endgameFeature(enemyDist)
369         );
370     }
371
372     default EvaluationLearningSample buildLearningSample(Board
        before, Move move, int playerId,

```

```

373                                     int
                                     size
                                     ,
                                     int
                                     wallsLeft1
                                     ,
                                     int
                                     wallsLeft2
                                     ) {
374     if (move == null) {
375         return null;
376     }
377
378     EvaluationFeatures beforeFeatures = evaluationFeatures(
        before, playerId, size, wallsLeft1, wallsLeft2);
379     Board after = before.copy();
380     applyMove(after, move);
381
382     int newWallsLeft1 = wallsLeft1;
383     int newWallsLeft2 = wallsLeft2;
384     if (move.getMoveType() == MoveType.PLACE_WALL) {
385         if (move.getPlayerId() == 1) {
386             --newWallsLeft1;
387         } else {
388             --newWallsLeft2;
389         }
390     }
391
392     EvaluationFeatures afterFeatures = evaluationFeatures(
        after, playerId, size, newWallsLeft1, newWallsLeft2)
        ;
393     return new EvaluationLearningSample(playerId,
        beforeFeatures, afterFeatures);
394 }
395
396 default int endgameFeature(int myDist) {
397     if (myDist <= 1) {
398         return 10;

```

```

399     }
400     if (myDist == 2) {
401         return 4;
402     }
403     if (myDist == 3) {
404         return 1;
405     }
406     return 0;
407 }
408
409 default int calculateShortestPath(Board board, Position
    start, int targetRow) {
410     Set<Position> visited = new HashSet<>();
411     Queue<Position> queue = new ArrayDeque<>();
412
413     queue.add(start);
414     visited.add(start);
415
416     int depth = 0;
417
418     while (!queue.isEmpty()) {
419         for (int k = queue.size(); k > 0; --k) {
420             Position curr = queue.poll();
421
422             if (curr.row() == targetRow) return depth;
423
424             for (Position next : board.getPathNeighbors(
                curr)) {
425                 if (visited.add(next)) {
426                     queue.add(next);
427                 }
428             }
429         }
430         depth++;
431     }
432     return 1000;
433 }
434
435 default long getTimeLimit(ComputerPlayerHardnessLevel level
    , int size) {

```

```

436         int multiplier = Math.max(1, size / GameSize.NORMAL.
            getAmountOfTilesPerSide());
437     return switch (level) {
438         case EASY -> 700L * multiplier;
439         case MEDIUM -> 1600L * multiplier;
440         case HARD -> 2800L * multiplier;
441     };
442 }
443
444 default String boardKey(Board board) {
445     StringBuilder key = new StringBuilder();
446     key.append(board.getPositionOfPlayer1()).append('/').
        append(board.getPositionOfPlayer2()).append('/');
447
448     board.getTiles().entrySet().stream()
449         .sorted(Comparator
450             .comparingInt((Map.Entry<Position, ?>
451                 entry) -> entry.getKey().row())
452             .thenComparingInt(entry -> entry.getKey()
453                 ().col()))
454         .forEach(entry -> {
455             key.append(entry.getKey());
456             key.append(entry.getValue().
457                 isLeftMovementAvailable() ? '1' : '0');
458             key.append(entry.getValue().
459                 isForwardMovementAvailable() ? '1' : '0'
460                 );
461             key.append(entry.getValue().
462                 isRightMovementAvailable() ? '1' : '0');
463             key.append(entry.getValue().
464                 isBackwardsMovementAvailable() ? '1' : '
465                 0');
466         });
467
468     return key.toString();
469 }
470 }

```

Реализация случайного алгоритма

Листинг Б.1 — Класс RandomAlgorithm

```

1 public class RandomAlgorithm implements Algorithm {
2     private static final Random random = new Random();
3     private AlgorithmReport lastReport;
4     private List<Position> recentPositions = List.of();
5
6     @Override
7     public ComputerAlgorithmType getType() {
8         return ComputerAlgorithmType.RANDOM;
9     }
10
11    @Override
12    public AlgorithmReport getLastReport() {
13        return lastReport;
14    }
15
16    @Override
17    public void setRecentPositions(List<Position>
18        recentPositions) {
19        this.recentPositions = recentPositions == null ? List.
20            of() : List.copyOf(recentPositions);
21    }
22
23    @Override
24    public Move getMove(Board board,
25        ComputerPlayerHardnessLevel hardnessLevel, int playerId,
26        int wallsLeft1, int wallsLeft2) {
27        long startedAt = System.currentTimeMillis();
28        int amountOfWallsLeft = playerId == 1 ? wallsLeft1 :
29            wallsLeft2;
30        List<Move> moves = getMoves(board, playerId,
31            amountOfWallsLeft);
32
33        if (moves.isEmpty()) return null;
34    }
35 }

```



```

29     Move tacticalMove = findEndgameMove(board, playerId,
      amountOfWallsLeft);
30     if (tacticalMove != null) {
31         lastReport = new AlgorithmReport(getType().
      getDescription(), tacticalMove,
32             evaluateAfterMove(board, tacticalMove,
      playerId, wallsLeft1, wallsLeft2),
33             0, 1, 1, 0, 0,
34             System.currentTimeMillis() - startedAt,
35             "Случайный_алгоритм_применил_эндшпильное_пр
      авило_перед_случайным_выбором"");
36         return tacticalMove;
37     }
38
39     Move result;
40     switch (hardnessLevel) {
41         case MEDIUM -> {
42             moves.sort((a, b) -> Integer.compare(
43                 evaluateAfterMove(board, b, playerId,
44                     wallsLeft1, wallsLeft2),
45                 evaluateAfterMove(board, a, playerId,
46                     wallsLeft1, wallsLeft2)
47             ));
48             int half = moves.size() / 2 + 1;
49             result = moves.get(random.nextInt(half));
50         }
51         case HARD -> {
52             moves.sort((a, b) -> Integer.compare(
53                 evaluateAfterMove(board, b, playerId,
54                     wallsLeft1, wallsLeft2),
55                 evaluateAfterMove(board, a, playerId,
56                     wallsLeft1, wallsLeft2)
57             ));
58             int top = Math.min(3, moves.size());
59             result = moves.get(random.nextInt(top));
60         }
61         default -> result = moves.get(random.nextInt(moves.
      size()));
62     }
63     lastReport = new AlgorithmReport(getType().

```

```

        getDescription(), result,
60         evaluateAfterMove(board, result, playerId,
            wallsLeft1, wallsLeft2),
61         1, moves.size(), moves.size(), 0, 0,
62         System.currentTimeMillis() - startedAt,
63         describeHardness(hardnessLevel));
64     return result;
65 }
66
67 private String describeHardness(ComputerPlayerHardnessLevel
    hardnessLevel) {
68     return switch (hardnessLevel) {
69         case EASY -> "Случайный_выбор_среди_всех_допустимых
            _ходов";
70         case MEDIUM -> "Случайный_выбор_из_верхней_половины
            _ходов_по_функции_оценки";
71         case HARD -> "Случайный_выбор_из_трех_лучших_ходов_
            по_функции_оценки";
72     };
73 }
74
75 private int evaluateAfterMove(Board board, Move move, int
    playerId, int wallsLeft1, int wallsLeft2) {
76     Board copy = board.copy();
77     applyMove(copy, move);
78     int size = (int) Math.sqrt(copy.getTiles().size());
79     int newWallsLeft1 = wallsLeft1;
80     int newWallsLeft2 = wallsLeft2;
81     if (move.getMoveType() == MoveType.PLACE_WALL) {
82         if (move.getPlayerId() == 1) {
83             --newWallsLeft1;
84         } else {
85             --newWallsLeft2;
86         }
87     }
88     return evaluate(copy, playerId, size, newWallsLeft1,
        newWallsLeft2)
89         + movementPreference(move, board, playerId,
            size, recentPositions);
90 }

```


Реализация алгоритма MiniMax

Листинг В.1 — Класс MinimaxAlgorithm

```

1 public class MinimaxAlgorithm implements Algorithm {
2     /**
3      * уже посчитанные ранее позиции
4      */
5     private final Map<String, Integer> cache = new HashMap<>();
6     private AlgorithmReport lastReport;
7     private long nodesVisited;
8     private long consideredMoves;
9     private List<Position> recentPositions = List.of();
10
11     @Override
12     public ComputerAlgorithmType getType() {
13         return ComputerAlgorithmType.MINIMAX;
14     }
15
16     @Override
17     public AlgorithmReport getLastReport() {
18         return lastReport;
19     }
20
21     @Override
22     public void setRecentPositions(List<Position>
23         recentPositions) {
24         this.recentPositions = recentPositions == null ? List.
25             of() : List.copyOf(recentPositions);
26     }
27
28     /**
29      * пока есть время, итеративно увеличиваем глубину поиска р
30      * ешения
31      * если время кончилось -> отдаем лучшее, что нашли
32      * MiniMax оставлен как честный перебор без alpha-beta отсе
33      * чений
34      */

```

```

31  @Override
32  public Move getMove(Board board,
    ComputerPlayerHardnessLevel hardnessLevel, int playerId,
    int wallsLeft1, int wallsLeft2) {
33      long startedAt = System.currentTimeMillis();
34      int size = (int) Math.sqrt(board.getTiles().size());
35      long timeLimit = getTimeLimit(hardnessLevel, size);
36      long endTime = System.currentTimeMillis() + timeLimit;
37      int maxDepth = getMaxDepth(hardnessLevel, size);
38
39      cache.clear();
40      nodesVisited = 0;
41      consideredMoves = 0;
42
43      int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
44      Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
45      if (tacticalMove != null) {
46          int tacticalScore = scoreMoveAfterApply(board,
            tacticalMove, playerId, size, wallsLeft1,
            wallsLeft2);
47          lastReport = new AlgorithmReport(
48              getType().getDescription(),
49              tacticalMove,
50              tacticalScore,
51              0,
52              nodesVisited,
53              1,
54              0,
55              0,
56              System.currentTimeMillis() - startedAt,
57              "MiniMax_применил_эндшпильное_правило:_неме
                дленная_победа_или_срочная_блокировка"
58          );
59          return tacticalMove;
60      }
61
62      Move best = null;
63      int bestScore = 0;

```

```

64     int reachedDepth = 0;
65     for (int depth = 1; depth <= maxDepth; depth++) {
66         if (System.currentTimeMillis() > endTime) {
67             break;
68         }
69         SearchResult result = search(board, depth, playerId
70             , size, wallsLeft1, wallsLeft2, endTime);
71         if (result.move() != null) {
72             best = result.move();
73             bestScore = result.score();
74             reachedDepth = depth;
75         }
76     }
77     lastReport = new AlgorithmReport(
78         getType().getDescription(),
79         best,
80         bestScore,
81         reachedDepth,
82         nodesVisited,
83         consideredMoves,
84         0,
85         0,
86         System.currentTimeMillis() - startedAt,
87         "MiniMax_перебрал_дерево_без_alpha-beta_отсечен
            ий"
            + " ;_лимит_времени:_ " + timeLimit + "_м
            с"
88     );
89     return best;
90 }
91
92 private int getMaxDepth(ComputerPlayerHardnessLevel level,
93     int size) {
94     boolean smallBoard = size <= GameSize.SMALL.
95         getAmountOfTilesPerSide();
96     boolean largeBoard = size > GameSize.NORMAL.
97         getAmountOfTilesPerSide();
98     return switch (level) {
99         case EASY -> 3;
100        case MEDIUM -> smallBoard ? 5 : 4;

```

```

98         case HARD -> largeBoard ? 4 : 5;
99     };
100 }
101
102 @Override
103 public long getTimeLimit(ComputerPlayerHardnessLevel level,
104     int size) {
105     boolean largeBoard = size > GameSize.NORMAL.
106         getAmountOfTilesPerSide();
107     return switch (level) {
108         case EASY -> largeBoard ? 900L : 700L;
109         case MEDIUM -> largeBoard ? 2200L : 1800L;
110         case HARD -> largeBoard ? 4200L : 3600L;
111     };
112 }
113
114 /**
115  * генерируем всевозможные ходы, сортируем по быстрой оценк
116  * е,
117  * отбираем ограниченное количество и прогоняем через
118  * minimax
119  */
120
121 private SearchResult search(Board board, int depth, int
122     playerId, int size,
123     int wallsLeft1, int wallsLeft2,
124     long endTime) {
125     int currentWalls = playerId == 1 ? wallsLeft1 :
126         wallsLeft2;
127     List<Move> moves = orderedLimitedMoves(
128         board,
129         getMoves(board, playerId, currentWalls),
130         playerId,
131         size,
132         wallsLeft1,
133         wallsLeft2
134     );
135     consideredMoves += moves.size();
136
137     Move bestMove = null;
138     int bestValue = Integer.MIN_VALUE;

```

```

131     for (Move move : moves) {
132         if (System.currentTimeMillis() > endTime) {
133             return new SearchResult(bestMove, bestValue);
134         }
135
136         Board copy = board.copy();
137         applyMove(copy, move);
138
139         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
140         if (move.getMoveType() == MoveType.PLACE_WALL) {
141             if (playerId == 1) {
142                 --newWallsLeft1;
143             } else {
144                 --newWallsLeft2;
145             }
146         }
147
148         int value = minimax(copy, depth - 1, false,
149             playerId, size, newWallsLeft1,
150             newWallsLeft2, endTime);
151         value += movementPreference(move, board, playerId,
152             size, recentPositions);
153         if (value > bestValue) {
154             bestValue = value;
155             bestMove = move;
156         }
157     }
158     return new SearchResult(bestMove, bestValue);
159 }
160
161 /**
162  * если max = true (наш ход) - максимизируем оценку
163  * если max = false (ход противника) - минимизируем
164  */
165 private int minimax(Board board, int depth, boolean
    maximizing,
166
167     int playerId, int size,
168     int wallsLeft1, int wallsLeft2, long
    endTime) {

```



```

166         nodesVisited++;
167         if (System.currentTimeMillis() > endTime) {
168             return evaluate(board, playerId, size, wallsLeft1,
                wallsLeft2);
169         }
170         String key = boardKey(board) + "/" + depth + "/" +
            maximizing + "/" + wallsLeft1 + "/" + wallsLeft2;
171         if (cache.containsKey(key)) {
172             return cache.get(key);
173         }
174
175         Integer terminal = terminalScore(board, playerId, size,
            depth);
176         if (terminal != null) {
177             cache.put(key, terminal);
178             return terminal;
179         }
180
181         if (depth == 0) {
182             int val = evaluate(board, playerId, size,
                wallsLeft1, wallsLeft2);
183             cache.put(key, val);
184             return val;
185         }
186
187         int current = maximizing ? playerId : (3 - playerId);
188         int walls = current == 1 ? wallsLeft1 : wallsLeft2;
189         List<Move> moves = orderedLimitedMoves(
190             board,
191             getMoves(board, current, walls),
192             playerId,
193             size,
194             wallsLeft1,
195             wallsLeft2
196         );
197         consideredMoves += moves.size();
198
199         int best = maximizing ? Integer.MIN_VALUE : Integer.
            MAX_VALUE;
200         for (Move move : moves) {

```

```

201         Board copy = board.copy();
202         applyMove(copy, move);
203
204         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
205         if (move.getMoveType() == MoveType.PLACE_WALL) {
206             if (current == 1) {
207                 --newWallsLeft1;
208             } else {
209                 --newWallsLeft2;
210             }
211         }
212
213         int val = minimax(copy, depth - 1, !maximizing,
214             playerId, size, newWallsLeft1,
                newWallsLeft2, endTime);
215         if (maximizing) {
216             best = Math.max(best, val);
217         } else {
218             best = Math.min(best, val);
219         }
220     }
221     cache.put(key, best);
222     return best;
223 }
224
225 /**
226  * ограничение ширины перебора, чтобы обычный MiniMax не за
227  висал на большом поле
228  */
229 private List<Move> orderedLimitedMoves(Board board, List<
    Move> moves, int playerId, int size,
                int wallsLeft1, int
                wallsLeft2) {
230     List<Move> ordered = new ArrayList<>(moves);
231     ordered.sort((a, b) -> Integer.compare(
232         scoreMoveForOrdering(board, b, playerId, size,
            wallsLeft1, wallsLeft2),
233         scoreMoveForOrdering(board, a, playerId, size,
            wallsLeft1, wallsLeft2)

```

```

234         ));
235         if (ordered.size() <= 16) {
236             return ordered;
237         }
238         return ordered.subList(0, 16);
239     }
240
241     private int scoreMoveForOrdering(Board board, Move move,
242         int playerId, int size, int wallsLeft1, int wallsLeft2)
243     {
244         return scoreMoveAfterApply(board, move, playerId, size,
245             wallsLeft1, wallsLeft2)
246             + movementPreference(move, board, playerId,
247                 size, recentPositions);
248     }
249
250     private int scoreMoveAfterApply(Board board, Move move, int
251         playerId, int size, int wallsLeft1, int wallsLeft2) {
252         Board copy = board.copy();
253         applyMove(copy, move);
254         int newWallsLeft1 = wallsLeft1;
255         int newWallsLeft2 = wallsLeft2;
256         if (move.getMoveType() == MoveType.PLACE_WALL) {
257             if (move.getPlayerId() == 1) {
258                 --newWallsLeft1;
259             } else {
260                 --newWallsLeft2;
261             }
262         }
263         return evaluate(copy, playerId, size, newWallsLeft1,
264             newWallsLeft2);
265     }
266
267     private record SearchResult(Move move, int score) {
268     }
269 }

```

Реализация алгоритма AlphaBeta

Листинг Г.1 — Класс AlphaBetaAlgorithm

```

1 public class AlphaBetaAlgorithm implements Algorithm {
2     /**
3      * transposition table хранит оценку позиции вместе с глуби-
4      * ной и типом границы
5      */
6     private final Map<String, TranspositionEntry>
7         transpositionTable = new HashMap<>();
8     /**
9      * ходы, которые часто приводят к хорошим отсечениям
10     */
11     private final Map<String, Integer> goodMoves = new HashMap
12         <>();
13     /**
14      * лучшие ходы, которые приводят к  $\beta \leq \alpha$ 
15     */
16     private final Move[][] bestMoves = new Move[50][2];
17     private AlgorithmReport lastReport;
18     private long nodesVisited;
19     private long consideredMoves;
20     private long cutoffs;
21     private long tableHits;
22     private List<Position> recentPositions = List.of();
23
24     @Override
25     public ComputerAlgorithmType getType() {
26         return ComputerAlgorithmType.ALPHABETA;
27     }
28
29     @Override
30     public AlgorithmReport getLastReport() {
31         return lastReport;
32     }
33
34     @Override

```

```

32     public void setRecentPositions(List<Position>
        recentPositions) {
33         this.recentPositions = recentPositions == null ? List.
            of() : List.copyOf(recentPositions);
34     }
35
36     /**
37      * пока есть время, итеративно увеличиваем глубину поиска р
        ешения
38      * если время кончилось -> отдаем лучшее, что нашли
39      * AlphaBeta ищет глубже MiniMax за счет сортировки ходов и
        отсечений
40      */
41     @Override
42     public Move getMove(Board board,
        ComputerPlayerHardnessLevel level, int playerId, int
        wallsLeft1, int wallsLeft2) {
43         long startedAt = System.currentTimeMillis();
44         int size = (int) Math.sqrt(board.getTiles().size());
45         int maxDepth = getMaxDepth(level, size);
46         long timeLimit = getTimeLimit(level, size);
47         long endTime = System.currentTimeMillis() + timeLimit;
48
49         transpositionTable.clear();
50         nodesVisited = 0;
51         consideredMoves = 0;
52         cutoffs = 0;
53         tableHits = 0;
54
55         int currentWalls = playerId == 1 ? wallsLeft1 :
            wallsLeft2;
56         Move tacticalMove = findEndgameMove(board, playerId,
            currentWalls);
57         if (tacticalMove != null) {
58             int tacticalScore = scoreMoveAfterApply(board,
                tacticalMove, playerId, size, wallsLeft1,
                wallsLeft2);
59             lastReport = new AlgorithmReport(
60                 getType().getDescription(),
61                 tacticalMove,

```

```

62         tacticalScore,
63         0,
64         nodesVisited,
65         1,
66         cutoffs,
67         tableHits,
68         System.currentTimeMillis() - startedAt,
69         "AlphaBeta_применил_эндшпильное_правило:_не
           медленная_победа_или_срочная_блокировка"
70     );
71     return tacticalMove;
72 }
73
74 Move bestMove = null;
75 int bestScore = 0;
76 int reachedDepth = 0;
77 for (int depth = 1; depth <= maxDepth; ++depth) {
78     if (System.currentTimeMillis() > endTime) {
79         break;
80     }
81     SearchResult result = search(board, depth, playerId
           , size, wallsLeft1, wallsLeft2, endTime);
82     if (result.move() != null) {
83         bestMove = result.move();
84         bestScore = result.score();
85         reachedDepth = depth;
86     }
87 }
88 lastReport = new AlgorithmReport(
89     getType().getDescription(),
90     bestMove,
91     bestScore,
92     reachedDepth,
93     nodesVisited,
94     consideredMoves,
95     cutoffs,
96     tableHits,
97     System.currentTimeMillis() - startedAt,
98     "AlphaBeta_использует_сортировку_ходов,_beta_<=
           _alpha_отсечения_и_transposition_table"

```

```

99         + "_c_depth-bound_flags;_попаданий_в_та
           блицу:_" + tableHits
100       + ";_лимит_времени:_" + timeLimit + "_м
           с"

101     );
102     return bestMove;
103 }
104
105 private int getMaxDepth(ComputerPlayerHardnessLevel level,
106     int size) {
107     boolean smallBoard = size <= GameSize.SMALL.
108         getAmountOfTilesPerSide();
109     boolean largeBoard = size > GameSize.NORMAL.
110         getAmountOfTilesPerSide();
111     return switch (level) {
112         case EASY -> smallBoard ? 5 : 4;
113         case MEDIUM -> smallBoard ? 7 : 6;
114         case HARD -> largeBoard ? 7 : 8;
115     };
116 }
117
118 @Override
119 public long getTimeLimit(ComputerPlayerHardnessLevel level,
120     int size) {
121     boolean largeBoard = size > GameSize.NORMAL.
122         getAmountOfTilesPerSide();
123     boolean smallBoard = size <= GameSize.SMALL.
124         getAmountOfTilesPerSide();
125     return switch (level) {
126         case EASY -> smallBoard ? 1000L : 1300L;
127         case MEDIUM -> largeBoard ? 4800L : 3600L;
128         case HARD -> largeBoard ? 9500L : 7600L;
129     };
130 }
131
132 /**
133  * генерируем всевозможные ходы, сортируем и отбираем из ни
134  * х топ кандидатов
135  * потом прогоняем через alpha-beta функцию оценки
136  */

```

```

130     private SearchResult search(Board board, int depth, int
        playerId, int size,
131         int wallsLeft1, int wallsLeft2,
            long endTime) {
132     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
133     List<Move> moves = new ArrayList<>(getMoves(board,
        playerId, currentWalls).stream()
134         .filter(move -> isRootMoveLegal(board, move,
            playerId, currentWalls))
135         .toList());
136     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, 0, null);
137     if (moves.size() > 36) {
138         moves = moves.subList(0, 36);
139     }
140     consideredMoves += moves.size();
141
142     Move bestMove = null;
143     int best = Integer.MIN_VALUE;
144     for (Move move : moves) {
145         if (System.currentTimeMillis() > endTime) {
146             break;
147         }
148         Board copy = board.copy();
149         applyMove(copy, move);
150
151         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
152         if (move.getMoveType() == MoveType.PLACE_WALL) {
153             if (playerId == 1) {
154                 --newWallsLeft1;
155             } else {
156                 --newWallsLeft2;
157             }
158         }
159         int val = alphaBeta(copy, depth - 1, Integer.
            MIN_VALUE, Integer.MAX_VALUE, false,
160             playerId, size, newWallsLeft1,
                newWallsLeft2, endTime, 1);

```



```

161         val += rootMovePreference(move, board, playerId,
162                                     size);
163         if (val > best) {
164             best = val;
165             bestMove = move;
166         }
167     }
168     return new SearchResult(bestMove, best);
169 }
170 /**
171  * если max = true (наш ход) - максимизируем оценку - резул
172  * ьтат в alpha (нижней границе)
173  * если max = false (ход противника) - минимизируем - резул
174  * ьтат в beta (верхней границе)
175  * если beta <= alpha -> можно не исследовать дальше, отсеч
176  * ение
177  */
178 private int alphaBeta(Board board, int depth, int alpha,
179                       int beta, boolean max,
180                       int playerId, int size, int
181                       wallsLeft1, int wallsLeft2,
182                       long endTime, int ply) {
183     nodesVisited++;
184     if (System.currentTimeMillis() > endTime) {
185         return evaluate(board, playerId, size, wallsLeft1,
186                        wallsLeft2);
187     }
188     String key = boardKey(board) + "/" + max + "/" +
189                 wallsLeft1 + "/" + wallsLeft2;
190     int originalAlpha = alpha;
191     int originalBeta = beta;
192
193     TranspositionEntry cached = transpositionTable.get(key)
194     ;
195     if (cached != null && cached.depth >= depth) {
196         if (cached.bound == Bound.EXACT) {
197             tableHits++;
198             return cached.score;
199         }
200     }

```

```

192         if (cached.bound == Bound.LOWER) {
193             alpha = Math.max(alpha, cached.score);
194         } else if (cached.bound == Bound.UPPER) {
195             beta = Math.min(beta, cached.score);
196         }
197         if (alpha >= beta) {
198             tableHits++;
199             return cached.score;
200         }
201     }
202
203     Integer terminal = terminalScore(board, playerId, size,
204                                     depth);
205     if (terminal != null) {
206         transpositionTable.put(key, new TranspositionEntry(
207             depth, terminal, Bound.EXACT, null));
208         return terminal;
209     }
210
211     if (depth == 0) {
212         if (isNoisyPosition(board, playerId, size)) {
213             int calmScore = quiescence(board, alpha, beta,
214                                     max, playerId, size,
215                                     wallsLeft1, wallsLeft2, endTime, 2);
216             transpositionTable.put(key, new
217                 TranspositionEntry(depth, calmScore, Bound.
218                     EXACT, null));
219             return calmScore;
220         }
221         int eval = evaluate(board, playerId, size,
222                             wallsLeft1, wallsLeft2);
223         transpositionTable.put(key, new TranspositionEntry(
224             depth, eval, Bound.EXACT, null));
225         return eval;
226     }
227
228     int current = max ? playerId : 3 - playerId;
229     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
230
231     List<Move> moves = getMoves(board, current, walls);

```

```

225     Move tableBestMove = cached == null ? null : cached.
        bestMove;
226     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, ply, tableBestMove);
227     if (moves.size() > 36) {
228         moves = moves.subList(0, 36);
229     }
230     consideredMoves += moves.size();
231
232     int best = max ? Integer.MIN_VALUE : Integer.MAX_VALUE;
233     Move bestMove = null;
234
235     for (Move move : moves) {
236         Board copy = board.copy();
237         applyMove(copy, move);
238
239         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
240         if (move.getMoveType() == MoveType.PLACE_WALL) {
241             if (current == 1) {
242                 --newWallsLeft1;
243             } else {
244                 --newWallsLeft2;
245             }
246         }
247
248         int val = alphaBeta(copy, depth - 1, alpha, beta, !
            max,
249             playerId, size, newWallsLeft1,
                newWallsLeft2, endTime, ply + 1);
250
251         if (max) {
252             if (val > best) {
253                 best = val;
254                 bestMove = move;
255             }
256             alpha = Math.max(alpha, val);
257         } else {
258             if (val < best) {
259                 best = val;

```

```

260             bestMove = move;
261         }
262         beta = Math.min(beta, val);
263     }
264
265     if (beta <= alpha) {
266         cutoffs++;
267         updateBestMoves(move, ply);
268         updateGoodMoves(move);
269         break;
270     }
271 }
272
273 Bound bound = Bound.EXACT;
274 if (best <= originalAlpha) {
275     bound = Bound.UPPER;
276 } else if (best >= originalBeta) {
277     bound = Bound.LOWER;
278 }
279 transpositionTable.put(key, new TranspositionEntry(
280     depth, best, bound, bestMove));
281 return best;
282 }
283
284 private int quiescence(Board board, int alpha, int beta,
285     boolean max,
286     int playerId, int size, int
287     wallsLeft1, int wallsLeft2,
288     long endTime, int extensionDepth) {
289     nodesVisited++;
290     int standPat = evaluate(board, playerId, size,
291         wallsLeft1, wallsLeft2);
292     if (extensionDepth == 0 || System.currentTimeMillis() >
293         endTime) {
294         return standPat;
295     }
296
297     if (max) {
298         if (standPat >= beta) {
299             return beta;
300         }
301     }

```

```

295         }
296         alpha = Math.max(alpha, standPat);
297     } else {
298         if (standPat <= alpha) {
299             return alpha;
300         }
301         beta = Math.min(beta, standPat);
302     }
303
304     int current = max ? playerId : 3 - playerId;
305     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
306     List<Move> moves = getQuiescenceMoves(board, current,
307         walls, playerId, size);
308     if (moves.isEmpty()) {
309         return standPat;
310     }
311     orderMoves(moves, board, playerId, size, wallsLeft1,
312         wallsLeft2, 0, null);
313
314     for (Move move : moves) {
315         if (System.currentTimeMillis() > endTime) {
316             break;
317         }
318         Board copy = board.copy();
319         applyMove(copy, move);
320
321         int newWallsLeft1 = wallsLeft1;
322         int newWallsLeft2 = wallsLeft2;
323         if (move.getMoveType() == MoveType.PLACE_WALL) {
324             if (current == 1) {
325                 --newWallsLeft1;
326             } else {
327                 --newWallsLeft2;
328             }
329         }
330
331         int value = quiescence(copy, alpha, beta, !max,
332             playerId, size,
333             newWallsLeft1, newWallsLeft2, endTime,
334             extensionDepth - 1);

```

```

331         if (max) {
332             if (value >= beta) {
333                 cutoffs++;
334                 return beta;
335             }
336             alpha = Math.max(alpha, value);
337         } else {
338             if (value <= alpha) {
339                 cutoffs++;
340                 return alpha;
341             }
342             beta = Math.min(beta, value);
343         }
344     }
345     return max ? alpha : beta;
346 }
347
348 /**
349  * сортируем ходы
350  */
351 private void orderMoves(List<Move> moves, Board board, int
    playerId, int size,
352                        int wallsLeft1, int wallsLeft2, int
    ply, Move tableBestMove) {
353     moves.sort((a, b) -> Integer.compare(
354         scoreMove(b, board, playerId, size, wallsLeft1,
    wallsLeft2, ply, tableBestMove),
355         scoreMove(a, board, playerId, size, wallsLeft1,
    wallsLeft2, ply, tableBestMove)
356     ));
357 }
358
359 /**
360  * оцениваем ход для корректной сортировки с помощью функции
    и evaluate из класса Algorithm
361  * будет плюсом наличие этого хода в истории хороших или лу
    чших ходов
362  */
363 private int scoreMove(Move move, Board board, int playerId,
    int size,

```

```

364         int wallsLeft1, int wallsLeft2, int
           ply, Move tableBestMove) {
365     int score = 0;
366
367     if (move.equals(tableBestMove)) {
368         score += 20000;
369     }
370
371     if (ply < bestMoves.length) {
372         if (bestMoves[ply][0] != null && move.equals(
           bestMoves[ply][0])) {
373             score += 10000;
374         }
375         if (bestMoves[ply][1] != null && move.equals(
           bestMoves[ply][1])) {
376             score += 8000;
377         }
378     }
379
380     score += goodMoves.getOrDefault(move.toString(), 0) *
           2;
381     score += rootMovePreference(move, board, playerId, size
           );
382
383     score += scoreMoveAfterApply(board, move, playerId,
           size, wallsLeft1, wallsLeft2);
384
385     return score;
386 }
387
388 private int scoreMoveAfterApply(Board board, Move move, int
           playerId, int size, int wallsLeft1, int wallsLeft2) {
389     Board copy = board.copy();
390     applyMove(copy, move);
391     int newWallsLeft1 = wallsLeft1;
392     int newWallsLeft2 = wallsLeft2;
393     if (move.getMoveType() == MoveType.PLACE_WALL) {
394         if (move.getPlayerId() == 1) {
395             --newWallsLeft1;
396         } else {

```

```

397         --newWallsLeft2;
398     }
399 }
400     return evaluate(copy, playerId, size, newWallsLeft1,
        newWallsLeft2);
401 }
402
403 private int rootMovePreference(Move move, Board board, int
    playerId, int size) {
404     return movementPreference(move, board, playerId, size,
        recentPositions);
405 }
406
407 private boolean isRootMoveLegal(Board board, Move move, int
    playerId, int wallsLeft) {
408     if (move.getPlayerId() != playerId) {
409         return false;
410     }
411     if (move.getMoveType() == MoveType.MOVE_PLAYER) {
412         return board.getAvailableMoves(getCurrentPosition(
            board, playerId)).contains(move.getEndPosition()
        );
413     }
414     if (wallsLeft <= 0) {
415         return false;
416     }
417     return isWallPlaceable(board, move.getStartPosition(),
        move.getEndPosition())
        && isValidWall(board, move.getStartPosition(),
            move.getEndPosition());
418 }
419
420
421 /**
422  * сохраняем лучший ход
423  */
424 private void updateBestMoves(Move move, int ply) {
425     if (ply >= bestMoves.length || move.equals(bestMoves[
        ply][0])) {
426         return;
427     }

```



```

428         bestMoves[ply][1] = bestMoves[ply][0];
429         bestMoves[ply][0] = move;
430     }
431
432     /**
433      * добавляем хороший ход в историю
434      */
435     private void updateGoodMoves(Move move) {
436         goodMoves.merge(move.toString(), 1, Integer::sum);
437     }
438
439     private record SearchResult(Move move, int score) {
440     }
441
442     private enum Bound {
443         EXACT,
444         LOWER,
445         UPPER
446     }
447
448     private record TranspositionEntry(int depth, int score,
449         Bound bound, Move bestMove) {
450     }

```

Реализация алгоритма Monte Carlo

Листинг Д.1 — Класс MonteCarloAlgorithm

```

1 public class MonteCarloAlgorithm implements Algorithm {
2     /**
3      * коэффициент баланса exploration/exploitation (UCT)
4      * больше -> больше случайности, меньше -> больше жадности
5      */
6     private static final double C = 1.2;
7     private static final Random random = new Random();
8     private AlgorithmReport lastReport;
9     private List<Position> recentPositions = List.of();
10    private int rootPlayerId;
11
12    @Override
13    public ComputerAlgorithmType getType() {
14        return ComputerAlgorithmType.MONTECARLO;
15    }
16
17    @Override
18    public AlgorithmReport getLastReport() {
19        return lastReport;
20    }
21
22    @Override
23    public void setRecentPositions(List<Position>
24        recentPositions) {
25        this.recentPositions = recentPositions == null ? List.
26            of() : List.copyOf(recentPositions);
27    }
28
29    /**
30     * идем по узлам дерева:
31     * добавляем детей узлу
32     * играем случайную партию
33     * обновляем статистику входов в узел и побед из него
34     */

```

```

33  @Override
34  public Move getMove(Board board,
    ComputerPlayerHardnessLevel hardnessLevel, int playerId,
    int wallsLeft1, int wallsLeft2) {
35      long startedAt = System.currentTimeMillis();
36      int size = (int) Math.sqrt(board.getTiles().size());
37      long timeLimit = getTimeLimit(hardnessLevel, size);
38      long endTime = System.currentTimeMillis() + timeLimit;
39      int rolloutDepth = getRolloutDepth(hardnessLevel, size)
        ;
40      long iterationBudget = getIterationBudget(hardnessLevel
        , size);
41
42      long iterations = 0;
43      rootPlayerId = playerId;
44      int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
45      Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
46      if (tacticalMove != null) {
47          lastReport = new AlgorithmReport(
48              getType().getDescription(),
49              tacticalMove,
50              scoreMove(board, tacticalMove, playerId,
                wallsLeft1, wallsLeft2),
51              0,
52              0,
53              1,
54              0,
55              0,
56              System.currentTimeMillis() - startedAt,
57              "MCTS_применил_эндшпильное_правило_до_запус
                ка_симуляций"
58          );
59      return tacticalMove;
60  }
61
62      Node root = new Node(board.copy(), null, null, playerId
        , wallsLeft1, wallsLeft2);
63      while (System.currentTimeMillis() < endTime &&

```

```

        iterations < iterationBudget) {
64         Node node = select(root);
65         Node expanded = expand(node);
66         int winner = simulate(expanded, rolloutDepth,
            playerId, size);
67         backpropagate(expanded, winner, playerId);
68         iterations++;
69         if (root.visits > 300 && bestWinRate(root) > 0.97)
            {
70             break;
71         }
72     }
73
74     Node best = root.children.stream()
75         .max(Comparator.comparingDouble(this::
            robustChildScore))
76         .orElseThrow();
77     lastReport = new AlgorithmReport(
78         getType().getDescription(),
79         best.move,
80         evaluate(best.board, playerId, size, best.
            walls1, best.walls2),
81         rolloutDepth,
82         iterations,
83         root.children.size(),
84         0,
85         0,
86         System.currentTimeMillis() - startedAt,
87         "MCTS: _выбор_по_UCT, _rollout_c_epsilon-greedy_э
            вристикой, _немедленными_победами_и_тактическ
            ими_стенами"
88         + " _лимит_времени:_" + timeLimit + " _м
            с"
89         + " _бюджет_итераций:_" +
            iterationBudget
90     );
91     return best.move;
92 }
93
94 private int getRolloutDepth(ComputerPlayerHardnessLevel

```

```

    level, int size) {
95         return switch (level) {
96             case EASY -> size * 4;
97             case MEDIUM -> size * 7;
98             case HARD -> size * 10;
99         };
100     }
101
102     private long getIterationBudget(ComputerPlayerHardnessLevel
        level, int size) {
103         boolean largeBoard = size > GameSize.NORMAL.
            getAmountOfTilesPerSide();
104         return switch (level) {
105             case EASY -> largeBoard ? 380L : 520L;
106             case MEDIUM -> largeBoard ? 1300L : 1700L;
107             case HARD -> largeBoard ? 2600L : 3400L;
108         };
109     }
110
111     @Override
112     public long getTimeLimit(ComputerPlayerHardnessLevel level,
        int size) {
113         boolean largeBoard = size > GameSize.NORMAL.
            getAmountOfTilesPerSide();
114         return switch (level) {
115             case EASY -> largeBoard ? 1200L : 950L;
116             case MEDIUM -> largeBoard ? 3800L : 2900L;
117             case HARD -> largeBoard ? 7600L : 6200L;
118         };
119     }
120
121     /**
122      * идем вниз дерева по пути наибольшего UCT
123      */
124     private Node select(Node node) {
125         while (!node.children.isEmpty() && node.untriedMoves.
            isEmpty()) {
126             node = node.children.stream().max(Comparator.
                comparing(this::uct)).orElseThrow();
127         }

```

```

128         return node;
129     }
130
131     /**
132      * создаем всевозможные ходы из текущего узла дерева
133      */
134     private Node expand(Node node) {
135         if (node.untriedMoves.isEmpty() && !node.children.
136             isEmpty()) {
137             return node;
138         }
139
140         if (node.untriedMoves.isEmpty()) {
141             node.untriedMoves.addAll(getMoves(node.board, node.
142                 player, node.getWalls()));
143             node.untriedMoves.sort((a, b) -> Integer.compare(
144                 scoreMove(node.board, b, node.player, node.
145                     walls1, node.walls2),
146                 scoreMove(node.board, a, node.player, node.
147                     walls1, node.walls2)
148             ));
149         }
150
151         if (node.untriedMoves.isEmpty()) {
152             return node;
153         }
154
155         Move move = node.untriedMoves.remove(0);
156         Board copy = node.board.copy();
157         applyMove(copy, move);
158
159         int wallsLeft1 = node.walls1;
160         int wallsLeft2 = node.walls2;
161
162         if (move.getMoveType() == MoveType.PLACE_WALL) {
163             if (node.player == 1) {
164                 --wallsLeft1;
165             } else {
166                 --wallsLeft2;
167             }
168         }

```

```

164     }
165     Node child = new Node(copy, node, move, 3 - node.player
        , wallsLeft1, wallsLeft2);
166     node.children.add(child);
167     return child;
168 }
169
170 /**
171  * играем случайную партию из узла дерева
172  */
173 private int simulate(Node node, int maxSteps, int
    rootPlayer, int size) {
174     Board board = node.board.copy();
175     int currentPlayer = node.player;
176     int wallsLeft1 = node.walls1;
177     int wallsLeft2 = node.walls2;
178
179     for (int step = 0; step < maxSteps; step++) {
180         if (isWin(board, 1, size)) {
181             return 1;
182         }
183         if (isWin(board, 2, size)) {
184             return 2;
185         }
186
187         int walls = currentPlayer == 1 ? wallsLeft1 :
            wallsLeft2;
188         List<Move> moves = getMoves(board, currentPlayer,
            walls);
189         if (moves.isEmpty()) {
190             return 3 - currentPlayer;
191         }
192
193         Move best = pickRolloutMove(board, moves,
            currentPlayer, rootPlayer, size, wallsLeft1,
            wallsLeft2);
194
195         applyMove(board, best);
196
197         if (best.getMoveType() == MoveType.PLACE_WALL) {

```

```

198         if (currentPlayer == 1) {
199             --wallsLeft1;
200         } else {
201             --wallsLeft2;
202         }
203     }
204     currentPlayer = 3 - currentPlayer;
205 }
206
207     return evaluate(board, rootPlayer, size, wallsLeft1,
208         wallsLeft2) > 0 ? rootPlayer : (3 - rootPlayer);
209 }
210
211 /**
212  * поднимаемся вверх по дереву и обновляем количество входов
213  * в и побед
214  */
215
216 private void backpropagate(Node node, int winner, int
217     playerId) {
218     while (node != null) {
219         ++node.visits;
220         if (winner == playerId) {
221             ++node.wins;
222         }
223         node = node.parent;
224     }
225 }
226
227 /**
228  * UCT – оценка узла
229  * exploitation – доля побед
230  * exploration – исследованность узла
231  * heuristic – оценка через evaluate
232  */
233
234 private double uct(Node n) {
235     if (n.visits == 0) {
236         return Double.MAX_VALUE;
237     }
238
239     double exploitation = n.wins / n.visits;

```



```

235     double exploration = C * Math.sqrt(Math.log(n.parent.
        visits) / n.visits);
236     double heuristic = normalizedEvaluate(n.board, n.parent
        .player, n.walls1, n.walls2);
237
238     return exploitation + exploration + heuristic;
239 }
240
241 private Move pickRolloutMove(Board board, List<Move> moves,
    int currentPlayer, int rootPlayer,
242     int size, int wallsLeft1, int
        wallsLeft2) {
243     for (Move move : moves) {
244         if (move.getMoveType() == MoveType.MOVE_PLAYER
245             && move.getEndPosition().row() ==
                getTargetRow(currentPlayer, size)) {
246             return move;
247         }
248     }
249
250     int walls = currentPlayer == 1 ? wallsLeft1 :
        wallsLeft2;
251     if (walls > 0 && calculateShortestPath(board,
        getCurrentPosition(board, 3 - currentPlayer),
252         getTargetRow(3 - currentPlayer, size)) <= 2) {
253         Move tacticalWall = moves.stream()
254             .filter(move -> move.getMoveType() ==
                MoveType.PLACE_WALL)
255             .max(Comparator.comparingInt(move ->
                wallImpact(board, move, currentPlayer,
                    size)))
256             .orElse(null);
257         if (tacticalWall != null && wallImpact(board,
            tacticalWall, currentPlayer, size) > 0) {
258             return tacticalWall;
259         }
260     }
261
262     if (random.nextDouble() < 0.22) {
263         return moves.get(random.nextInt(moves.size()));
    
```

```

264     }
265
266     int sampleSize = Math.min(18, moves.size());
267     List<Move> sampled = new ArrayList<>(sampleSize);
268     Set<Integer> usedIndexes = new HashSet<>();
269     while (sampled.size() < sampleSize) {
270         int index = random.nextInt(moves.size());
271         if (usedIndexes.add(index)) {
272             sampled.add(moves.get(index));
273         }
274     }
275
276     sampled.sort((a, b) -> Integer.compare(
277         scoreRolloutMove(board, b, currentPlayer,
278             rootPlayer, size, wallsLeft1, wallsLeft2),
279         scoreRolloutMove(board, a, currentPlayer,
280             rootPlayer, size, wallsLeft1, wallsLeft2)
281     ));
282     int top = Math.max(1, Math.min(4, sampled.size()));
283     return sampled.get(random.nextInt(top));
284 }
285
286 private int scoreRolloutMove(Board board, Move move, int
287     currentPlayer, int rootPlayer,
288     int size, int wallsLeft1, int
289     wallsLeft2) {
290     Board tmp = board.copy();
291     applyMove(tmp, move);
292     int newWallsLeft1 = wallsLeft1;
293     int newWallsLeft2 = wallsLeft2;
294     if (move.getMoveType() == MoveType.PLACE_WALL) {
295         if (currentPlayer == 1) {
296             --newWallsLeft1;
297         } else {
298             --newWallsLeft2;
299         }
300     }
301
302     int score = evaluate(tmp, rootPlayer, size,
303         newWallsLeft1, newWallsLeft2);

```

```

299         if (currentPlayer != rootPlayer) {
300             score = -score;
301         }
302         if (currentPlayer == rootPlayer) {
303             score += movementPreference(move, board,
304                                     currentPlayer, size, recentPositions);
305         }
306         return score;
307     }
308     private int scoreMove(Board board, Move move, int playerId,
309                          int wallsLeft1, int wallsLeft2) {
310         Board copy = board.copy();
311         applyMove(copy, move);
312         int size = (int) Math.sqrt(copy.getTiles().size());
313         int newWallsLeft1 = wallsLeft1;
314         int newWallsLeft2 = wallsLeft2;
315         if (move.getMoveType() == MoveType.PLACE_WALL) {
316             if (move.getPlayerId() == 1) {
317                 --newWallsLeft1;
318             } else {
319                 --newWallsLeft2;
320             }
321         }
322         int preference = playerId == rootPlayerId
323             ? movementPreference(move, board, playerId,
324                                 size, recentPositions)
325             : 0;
326         return evaluate(copy, playerId, size, newWallsLeft1,
327                       newWallsLeft2) + preference;
328     }
329     private double normalizedEvaluate(Board board, int playerId
330                                     , int wallsLeft1, int wallsLeft2) {
331         int size = (int) Math.sqrt(board.getTiles().size());
332         return Math.tanh(evaluate(board, playerId, size,
333                                 wallsLeft1, wallsLeft2) / 500.0) * 0.15;
334     }
335     private double robustChildScore(Node node) {

```

```

333         if (node.visits == 0) {
334             return 0;
335         }
336         return node.visits + node.wins / node.visits;
337     }
338
339     /**
340      * выбор лучшего ребенка через статистику побед
341      */
342     private double bestWinRate(Node root) {
343         return root.children.stream().mapToDouble(n -> n.visits
344             == 0 ? 0 : n.wins / n.visits).max().orElse(0);
345     }
346
347     /**
348      * проверяем, достиг ли игрок нужной строки поля
349      */
350     public boolean isWin(Board board, int playerId, int size) {
351         int row = getCurrentPosition(board, playerId).row();
352         return (playerId == 1 && row == 0) || (playerId == 2 &&
353             row == size - 1);
354     }
355
356     /**
357      * узел дерева
358      */
359     static class Node {
360         Board board;
361         Node parent;
362         List<Node> children = new ArrayList<>();
363         List<Move> untriedMoves = new ArrayList<>();
364         Move move;
365
366         int visits = 0;
367         double wins = 0;
368
369         int player;
370         int walls1, walls2;
371
372         Node(Board board, Node parent, Move move, int player,

```

```

371         int w1, int w2) {
372             this.board = board;
373             this.parent = parent;
374             this.move = move;
375             this.player = player;
376             this.walls1 = w1;
377             this.walls2 = w2;
378         }
379
380     int getWalls() {
381         return player == 1 ? walls1 : walls2;
382     }
383 }

```

Реализация обучения весов функции оценки

Листинг E.1 — Класс EvaluationWeightsStore

```

1 public final class EvaluationWeightsStore {
2     private static final Path FILE = Path.of("data", "
        evaluation-weights.properties");
3     private static final Path HISTORY_FILE = Path.of("data", "
        evaluation-weights-history.csv");
4     private static EvaluationWeights current;
5
6     private EvaluationWeightsStore() {
7     }
8
9     public static synchronized EvaluationWeights current() {
10         if (current == null) {
11             current = load();
12         }
13         return current;
14     }
15
16     public static synchronized EvaluationTrainingUpdate
        learnFromGame(List<EvaluationLearningSample> samples,
17
18
19         EvaluationWeights beforeWeights = current();
20         if (samples.isEmpty() || winnerId == 0) {
21             return new EvaluationTrainingUpdate(false, winnerId

```

```

        , samples.size(), new int[7], beforeWeights,
        beforeWeights);
22     }
23
24     int[] gradient = new int[7];
25     for (EvaluationLearningSample sample : samples) {
26         int reward = sample.playerId() == winnerId ? 1 :
            -1;
27         EvaluationFeatures delta = sample.delta();
28         gradient[0] += reward * delta.pathAdvantage();
29         gradient[1] += reward * delta.myMobility();
30         gradient[2] += reward * delta.enemyMobilityPenalty
            ();
31         gradient[3] += reward * delta.wallAdvantage();
32         gradient[4] += reward * delta.progressAdvantage();
33         gradient[5] += reward * delta.myEndgame();
34         gradient[6] += reward * delta.enemyEndgamePenalty()
            ;
35     }
36
37     if (isZero(gradient)) {
38         return new EvaluationTrainingUpdate(false, winnerId
            , samples.size(), gradient, beforeWeights,
            beforeWeights);
39     }
40
41     current = beforeWeights.adjustedByGameGradient(gradient
        , 1, samples.size());
42     save(current);
43     appendHistory(source, winnerId, samples.size(),
        gradient, beforeWeights, current);
44     return new EvaluationTrainingUpdate(true, winnerId,
        samples.size(), gradient, beforeWeights, current);
45 }
46
47 private static EvaluationWeights load() {
48     if (!Files.exists(FILE)) {
49         EvaluationWeights defaults = EvaluationWeights.
            defaults();
50         save(defaults);

```

```

51         return defaults;
52     }
53
54     Properties properties = new Properties();
55     try (Reader reader = Files.newBufferedReader(FILE,
56         StandardCharsets.UTF_8)) {
57         properties.load(reader);
58         return new EvaluationWeights(
59             readInt(properties, "pathAdvantage",
60                 EvaluationWeights.defaults().
61                 pathAdvantage()),
62             readInt(properties, "myMobility",
63                 EvaluationWeights.defaults().myMobility
64                 ()),
65             readInt(properties, "enemyMobility",
66                 EvaluationWeights.defaults().
67                 enemyMobility()),
68             readInt(properties, "wallAdvantage",
69                 EvaluationWeights.defaults().
70                 wallAdvantage()),
71             readInt(properties, "progressAdvantage",
72                 EvaluationWeights.defaults().
73                 progressAdvantage()),
74             readInt(properties, "myEndgame",
75                 EvaluationWeights.defaults().myEndgame()
76                 ),
77             readInt(properties, "enemyEndgame",
78                 EvaluationWeights.defaults().
79                 enemyEndgame()),
80             readLong(properties, "samples", 0)
81         );
82     } catch (IOException | IllegalArgumentException e) {
83         EvaluationWeights defaults = EvaluationWeights.
84             defaults();
85         save(defaults);
86         return defaults;
87     }
88 }
89
90 private static void save(EvaluationWeights weights) {

```



```

75         try {
76             Files.createDirectories(FILE.getParent());
77             Files.writeString(FILE, serialize(weights),
78                               StandardCharsets.UTF_8);
79         } catch (IOException ignored) {
80             // Если файл недоступен, алгоритмы продолжают работать с
81             // весами в памяти.
82         }
83     }
84
85     private static String serialize(EvaluationWeights weights)
86     {
87         return """
88             # Learned evaluation weights for Quoridor AI.
89             # Файл можно редактировать вручную; при следующ
90             ем сохранении комментарии будут сохранены.
91             #
92             # pathAdvantage - вес разницы кратчайших путей:
93             # путь соперника минус путь ИИ.
94             # Чем больше значение, тем сильнее ИИ стремится
95             # сокращать свой путь и удлинять путь соперника.
96             pathAdvantage=%d
97
98             # myMobility - вес количества доступных ходов ф
99             ишкой для ИИ.
100            # Увеличивает ценность позиций, где у ИИ больше
101            вариантов движения.
102            myMobility=%d
103
104            # enemyMobility - штраф за количество доступных
105            ходов фишкой у соперника.
106            # Чем больше значение, тем сильнее ИИ старается
107            ограничивать мобильность противника.
108            enemyMobility=%d
109
110            # wallAdvantage - вес преимущества по оставшимс
111            я стенам: стены ИИ минус стены соперника.
112            # Помогает не тратить стены без необходимости и
113            ценить запас стен в эндшпиле.
114            wallAdvantage=%d

```

```

103
104     .....#_progressAdvantage_-_вес_продвижения_к_целевой
        ....._линии_относительно_соперника.
105     .....#_Нужен,_чтобы_алгоритм_не_зацикливался_на_равн
        .....ых_по_длине_пути_позициях.
106     .....progressAdvantage=%d
107
108     .....#_myEndgame_-_бонус_за_близость_ИИ_к_победной_л
        .....инии.
109     .....#_Работает_особенно_сильно,_когда_до_цели_остал
        .....ось_1-3_хода.
110     .....myEndgame=%d
111
112     .....#_enemyEndgame_-_штраф_за_близость_соперника_к_
        .....победной_линии.
113     .....#_Чем_больше_значение,_тем_охотнее_ИИ_срочно_бл
        .....окирует_финиш_соперника.
114     .....enemyEndgame=%d
115
116     .....#_samples_-_сколько_ходов_ИИ_участвовало_в_прин
        .....ятых_обновлениях_после_завершения_партий.
117     .....#_Весы_меняются_не_после_отдельного_хода,_а_пос
        .....ле_партии,_на_основании_победителя.
118     .....samples=%d
119     ....."".formatted(
120         weights.pathAdvantage(),
121         weights.myMobility(),
122         weights.enemyMobility(),
123         weights.wallAdvantage(),
124         weights.progressAdvantage(),
125         weights.myEndgame(),
126         weights.enemyEndgame(),
127         weights.samples()
128     );
129 }
130
131     private static void appendHistory(String source, int
        winnerId, int sampleCount, int[] gradient,
132                                     EvaluationWeights before,
        EvaluationWeights

```

```

after) {
133     try {
134         Files.createDirectories(HISTORY_FILE.getParent());
135         if (!Files.exists(HISTORY_FILE)) {
136             Files.writeString(HISTORY_FILE, historyHeader()
137                 , StandardCharsets.UTF_8);
138         }
139         Files.writeString(HISTORY_FILE, historyRow(source,
140             winnerId, sampleCount, gradient, before, after),
141             StandardCharsets.UTF_8, StandardOpenOption.
142                 APPEND);
143     } catch (IOException ignored) {
144         // История полезна для диплома, но недоступность файла не
145         // должна ломать обучение.
146     }
147 }
148
149 private static String historyHeader() {
150     return "timestamp,source,winnerId,sampleCount,"
151         + "gradientPath,gradientMyMobility,
152             gradientEnemyMobility,gradientWall,
153             gradientProgress,gradientMyEndgame,
154             gradientEnemyEndgame,"
155         + "beforePath,beforeMyMobility,
156             beforeEnemyMobility,beforeWall,
157             beforeProgress,beforeMyEndgame,
158             beforeEnemyEndgame,beforeSamples,"
159         + "afterPath,afterMyMobility,afterEnemyMobility
160             ,afterWall,afterProgress,afterMyEndgame,
161             afterEnemyEndgame,afterSamples\n";
162 }
163
164 private static String historyRow(String source, int
165     winnerId, int sampleCount, int[] gradient,
166     EvaluationWeights before,
167     EvaluationWeights after
168 ) {
169     return String.join(", ",
170         LocalDateTime.now().toString(),
171         escape(source),

```

```

157         Integer.toString(winnerId),
158         Integer.toString(sampleCount),
159         Integer.toString(gradient[0]),
160         Integer.toString(gradient[1]),
161         Integer.toString(gradient[2]),
162         Integer.toString(gradient[3]),
163         Integer.toString(gradient[4]),
164         Integer.toString(gradient[5]),
165         Integer.toString(gradient[6]),
166         Integer.toString(before.pathAdvantage()),
167         Integer.toString(before.myMobility()),
168         Integer.toString(before.enemyMobility()),
169         Integer.toString(before.wallAdvantage()),
170         Integer.toString(before.progressAdvantage()),
171         Integer.toString(before.myEndgame()),
172         Integer.toString(before.enemyEndgame()),
173         Long.toString(before.samples()),
174         Integer.toString(after.pathAdvantage()),
175         Integer.toString(after.myMobility()),
176         Integer.toString(after.enemyMobility()),
177         Integer.toString(after.wallAdvantage()),
178         Integer.toString(after.progressAdvantage()),
179         Integer.toString(after.myEndgame()),
180         Integer.toString(after.enemyEndgame()),
181         Long.toString(after.samples())
182     ) + "\n";
183 }
184
185 private static String escape(String value) {
186     String safe = value == null ? "" : value.replace("\\",
187         "\\");
188     return "\"" + safe + "\"";
189 }
190
191 private static int readInt(Properties properties, String
192     key, int fallback) {
193     return Integer.parseInt(properties.getProperty(key,
194         Integer.toString(fallback)));
195 }

```

```
194     private static long readLong(Properties properties, String
        key, long fallback) {
195         return Long.parseLong(properties.getProperty(key, Long.
            toString(fallback)));
196     }
197
198     private static boolean isZero(int[] values) {
199         for (int value : values) {
200             if (value != 0) {
201                 return false;
202             }
203         }
204         return true;
205     }
206 }
```